AI / ML FOUNDATIONS • STAGE 1 • COMPUTATIONAL LAYER

NumPy

Master Notes

PART 2

Computation on NumPy Arrays: Universal Functions + Aggregations

Based on: Python Data Science Handbook, Chapter 2 (Blocks 3 and 4)

Theory | Examples | Visual Outputs | Practice Problems | Complete Answer Key

WHAT IS INSIDE

- 1. The slowness of loops
- 2. Ufuncs and vectorization
- 3. Array arithmetic
- 4. Absolute value
- 5. Trigonometric functions
- 6. Exponents and logarithms
- 7. sum, min, max
- 8. Multidimensional aggregates and axis
- 9. mean, std, argmax and friends
- 10. Worked example: US presidents

Slow-paced notes: read, understand, then practise. Answers are at the very end.

Prepared for Abhijeet • October 2026

How to use these notes

These notes cover the two blocks you studied from the Python Data Science Handbook: **Block 3: Computation on NumPy Arrays: Universal Functions** and **Block 4: Aggregations: Min, Max, and Everything in Between**. Every topic follows the same order as before: the idea first, then a real-life picture, then code, then the exact output you will see in Colab. After every part you get practice problems. All correct answers are at the very end, in the Answer Key, with the real output of every program.

Because you said you cannot memorise the syntax of every function, each function carries a priority tag so you know where to spend your energy. Functions that the book mentions but that are not worth your time right now were left out on purpose (the list is in the Revision Zone).

Tag	Meaning	What to do
MUST KNOW	You will type this almost every day in AI/ML work.	Practise until your fingers know it.
GOOD TO KNOW	Used regularly, but not every day.	Know what it does and roughly how it looks.
LOOK UP	Rare, or easy to search when needed.	Remember that it exists. Do not memorise.

The coloured boxes you will meet:

ANALOGY	A real-life picture so the idea sticks.
MEMORY TRICK	A short trick to remember something forever.
INTERVIEW FLAG	The kind of question interviewers and viva examiners love to ask.
INTERVIEW POINT	Write or say this in an answer for full marks.
IMPORTANT	Pay attention: easy to get wrong.
COMMON TRAP	A mistake almost every beginner makes. NumPy never warns you about most of these.
TRY IT	Type this into Colab and watch it happen yourself.
WHY IT MATTERS	Where this idea shows up in real AI/ML work.
VERSION NOTE	Newer NumPy differs from the book in a small way.
NOTE	Extra detail that helps understanding.

The practice problem tags:

Tag	What you must do
PREDICT THE OUTPUT	Read the code and say exactly what Python will print. Do it in your head first, then run it.
FIND THE BUGS	The code looks fine but is broken or gives a wrong result. Find every mistake.
BUILD IT	Write the code yourself from a description.
THINK IT THROUGH	Reason about why. No memorising possible.
CALCULATE	Work out sizes and numbers step by step.
READ & DECODE	Read a line of NumPy and explain it in plain words.
THEORY	Interview-style written answers.

STUDY RULE

STUDY RULE. Do not read these notes like a story book only. Open Colab, type the examples, change one value at a time and watch what changes. NumPy is learnt with the fingers, not only the eyes.

All outputs in these notes were produced by running the code (NumPy 2.4.4), so what you see on the page is what you should see on your screen. Timings from `%timeit` are the ones printed in the book and will differ on your machine.

Your NumPy topic map

#	Topic	Why it matters in AI/ML	Priority
1	The slowness of loops	Explains why NumPy exists: never loop over data	MUST KNOW
2	Ufuncs and vectorization	The idea behind all fast ML code (and PyTorch)	MUST KNOW
3	Array arithmetic	Every formula in ML code is arithmetic on arrays	MUST KNOW
4	Absolute value	Mean Absolute Error, L1 regularisation	GOOD TO KNOW
5	Trigonometric functions	Positional encodings, similarity; float precision lesson	GOOD TO KNOW
6	Exponents and logarithms	Sigmoid, softmax, cross-entropy loss	MUST KNOW
7	sum, min and max	Summary values; built-in versus NumPy	MUST KNOW
8	Multidimensional aggregates and axis	Normalisation, per-sample and per-feature results	MUST KNOW
9	mean, std, argmax and friends	Predicted classes, spread, missing data	MUST KNOW
10	Worked example: US presidents	Reading summary statistics and a histogram	GOOD TO KNOW

Contents

Part 1: The Slowness of Loops . . . 6

1.1 Fast or slow: the one idea behind this whole chapter . . . 6
1.2 Why a Python loop is slow . . . 6
1.3 The book's example: reciprocals . . . 7
1.4 How to read %timeit and its units . . . 8
1.5 Side note: Python can be sped up in other ways . . . 8

Part 2: Introducing Ufuncs . . . 10

2.1 The cure: vectorized operations . . . 10
2.2 Same answer, a thousand times faster . . . 10
2.3 Ufuncs are flexible: three kinds of input . . . 11
2.4 Unary and binary ufuncs . . . 12
2.5 The golden rule . . . 12

Part 3: Array Arithmetic . . . 14

3.1 The basic operators . . . 14
3.2 Chaining and order of operations . . . 14
3.3 The hidden truth: operators are wrappers . . . 15
3.4 What type do you get back? . . . 15
3.5 Floor division and remainder: the traps . . . 16

Part 4: Absolute Value . . . 18

4.1 Three names, one ufunc . . . 18
4.2 Complex numbers: absolute value is the magnitude . . . 18
4.3 Why abs matters in ML: Mean Absolute Error . . . 19
4.4 A silent trap: the most negative integer . . . 19

Part 5: Trigonometric Functions . . . 21

5.1 Angles in NumPy are radians . . . 21
5.2 sin, cos and tan . . . 21
5.3 The surprise: why not exactly zero? . . . 22
5.4 The inverse functions . . . 22
5.5 Degrees and radians . . . 23
5.6 Where you meet trig in AI/ML . . . 23

Part 6: Exponents and Logarithms . . . 25

6.1 Exponentials . . . 25
6.2 Logarithms . . . 25
6.3 Exp and log undo each other, and the domain rules . . . 26
6.4 Precision tricks: expm1 and log1p . . . 26
6.5 The payoff: exp and log inside ML . . . 27

Part 7: Aggregations: sum, min and max . . . 29

7.1 What is an aggregation? . . . 29

7.2 Summing the values in an array . . . 29
7.3 The speed difference . . . 30
7.4 Minimum and maximum . . . 30
7.5 The trap: same name, different meaning . . . 31

Part 8: Multidimensional Aggregates and axis . . . 33

8.1 Default: aggregate everything . . . 33
8.2 The axis argument . . . 33
8.3 The confusing part, solved . . . 34
8.4 More than two dimensions . . . 35
8.5 keepdims: keep the collapsed axis as size 1 . . . 35
8.6 The ML pattern: samples by features . . . 35

Part 9: The Core Aggregation Functions . . . 38

9.1 The toolbox . . . 38
9.2 mean and median . . . 38
9.3 variance and standard deviation . . . 39
9.4 argmax and argmin: the position, not the value . . . 39
9.5 NaN-safe versions . . . 40

Part 10: Worked Example: Heights of US Presidents . . . 42

10.1 The data . . . 42
10.2 Summary statistics . . . 42
10.3 Drawing the histogram . . . 43
10.4 The mean-versus-median clue . . . 43

Part 11: Revision Zone . . . 45

11.1 Function cheat sheet . . . 45
11.2 Topic checklist . . . 46
11.3 Interview quick-fire: 20 questions with model answers . . . 46
11.4 The error radar: check these before you trust your output . . . 47
Answers: Part 1: The Slowness of Loops . . . 49
Answers: Part 2: Introducing Ufuncs . . . 50
Answers: Part 3: Array Arithmetic . . . 51
Answers: Part 4: Absolute Value . . . 53
Answers: Part 5: Trigonometric Functions . . . 54
Answers: Part 6: Exponents and Logarithms . . . 55
Answers: Part 7: sum, min and max . . . 56
Answers: Part 8: Multidimensional Aggregates and axis . . . 58
Answers: Part 9: The Core Aggregation Functions . . . 59
Answers: Part 10: Worked Example: Heights of US Presidents . . . 61
Answers: Final Mixed Paper (Blocks 3 and 4) . . . 62

PART 1

1 The Slowness of Loops

Block 3, first section: why Python loops over arrays are slow

IN THIS PART YOU WILL LEARN

- ✓ Why a Python loop over an array is slow, and what is really being repeated
- ✓ What dynamic typing costs on every single iteration of a loop
- ✓ How to measure speed with `%timeit` and read the units (s, ms, µs)
- ✓ Why the bottleneck is overhead and not the arithmetic itself

1.1 Fast or slow: the one idea behind this whole chapter

Until now we have met the nuts and bolts of NumPy: creating arrays, indexing, slicing, reshaping. From here on we see **why NumPy is so important** in the Python data-science world: it gives you an easy and flexible way to make computation on arrays of data **fast**.

Computation on NumPy arrays can be very fast, or it can be very slow. The key to making it fast is to use **vectorized operations**, which NumPy implements through **universal functions** (ufuncs). This chapter first shows the problem (the slowness of loops), then the cure (ufuncs), then the most useful ufuncs.

WHY IT MATTERS

Every ML calculation (a loss, an activation, a normalisation) is one small operation repeated over thousands or millions of numbers. If you write it as a Python loop it is painfully slow; written as a ufunc it runs about a thousand times faster. **Rule for life: avoid Python loops over data.**

1.2 Why a Python loop is slow

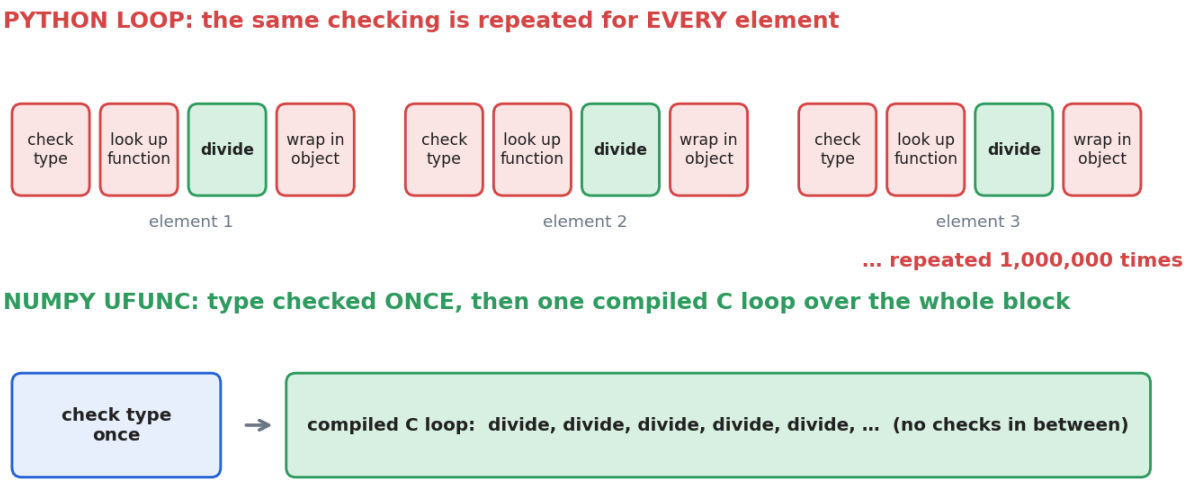
Python as you normally run it is called **CPython**. It is **dynamically typed**: a variable can hold an int now and a string a moment later. Flexible for you, but it means Python cannot know in advance what kind of object sits in `values[i]`, so it has to find out **every single time**.

Look at one line inside a loop, `output[i] = 1.0 / values[i]`. On every pass Python does four things:

Step	What Python does	Is it the real maths?
1	Looks at `values[i]` and works out its type (int? float? something else?)	No, overhead
2	Looks up which division routine belongs to that type (a dynamic lookup)	No, overhead
3	Runs the division	**Yes, the only real work**
4	Wraps the answer in a brand new Python object and stores it	No, overhead

ANALOGY

The cashier who re-reads the rulebook. A million identical items arrive at the till. A bad cashier reads the rulebook for every single item before scanning it. Scanning is quick; the rulebook-reading is what makes the queue crawl. A compiled language reads the rulebook **once** because it already knows all items are the same.



Where the time goes: three of the four boxes in a Python loop are pure overhead.

IMPORTANT

The slow part is NOT the arithmetic. It is the type checking and function lookup repeated on every cycle of the loop. The book says it directly: the bottleneck is not the operations themselves but the checks CPython must do.

1.3 The book's example: reciprocals

Imagine an array of values where we want the **reciprocal** (1 divided by the value) of each one. Someone from a C or Java background would write this loop:

PYTHON • type this in Colab

```
import numpy as np
rng = np.random.default_rng(seed=1701)

def compute_reciprocals(values):
    output = np.empty(len(values))
    for i in range(len(values)):
        output[i] = 1.0 / values[i]
    return output

values = rng.integers(1, 10, size=5)
print(values)
print(compute_reciprocals(values))
```

▶ OUTPUT

```
[9 4 1 3 8]
[0.11111111 0.25       1.         0.33333333 0.125     ]
```

It works, and for five numbers nobody notices a delay. Now make the input one million numbers and time it with IPython's `%timeit` magic (it works in Colab and Jupyter notebooks):

PYTHON • type this in Colab

```
big_array = rng.integers(1, 100, size=1000000)
%timeit compute_reciprocals(big_array)
```

▶ OUTPUT

```
2.61 s ± 192 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
```

This output is the figure printed in the book, measured on the author's computer. Your Colab will show a different number, but the same story.

2.61 seconds for one million divisions. A modern phone does billions of operations per second, so this is absurdly slow. That gap is exactly the per-element overhead from section 1.2.

INTERVIEW FLAG

"Why are Python loops slow on arrays? Is it because maths is slow in Python?"

Model answer: no. The arithmetic is quick. CPython is dynamically typed, so on every iteration it must check the object's type and look up the right function for it. With millions of iterations that repeated overhead dominates. Compiled code knows the type in advance and skips all of it.

1.4 How to read %timeit and its units

`%timeit` runs your line many times and reports the average and the spread. It is a notebook "magic" command, so it only works in Jupyter or Colab (outside a notebook use `time.perf_counter()`). The units are where people slip:

Unit	Name	Size	Where you meet it
s	second	1 s	the slow Python loop (2.61 s)
ms	millisecond	1 s = 1000 ms	the fast vectorized version (2.54 ms)
µs	microsecond	1 ms = 1000 µs	tiny NumPy calls (about 500 µs)

COMMON TRAP

Compare in the same unit. 2.61 s and 2.54 ms look almost equal if you ignore the units. Convert first: 2.61 s = 2610 ms, so the loop is about 2610 / 2.54 = **1000 times slower**.

1.5 Side note: Python can be sped up in other ways

The slowness of CPython is well known, so several projects try to fix it. You do not need to learn any of them now; just recognise the names.

Project	What it does	Priority
PyPy	A just-in-time compiled implementation of Python	LOOK UP
Cython	Converts Python code into compilable C code	LOOK UP
Numba	Converts snippets of Python code into fast LLVM bytecode	LOOK UP

NOTE

Each has strengths and weaknesses, but the book is clear that none of them has yet beaten the reach and popularity of standard CPython. NumPy stays the everyday answer for numeric work.

✎ PRACTICE PROBLEMS | Part 1: The Slowness of Loops

Q1 THEORY

[3 marks] Explain why a Python `for` loop over one million array elements is slow. Name what is repeated on every iteration.

Q2 THINK IT THROUGH

A friend says: "Doubling 10 million numbers in a loop is slow because the multiplication is slow, so buy a faster CPU." Is your friend right? What is actually slow?

Q3 READ & DECODE

In the line `output[i] = 1.0 / values[i]`, which part is the real arithmetic and which parts are overhead?

Q4 CALCULATE

A Python loop takes 2.61 s and the vectorized version takes 2.54 ms for the same job. How many times faster is the vectorized version? (Convert units first.)

Q5 PREDICT THE OUTPUT

Predict the output, and say why the result prints with decimal points even though the input is whole numbers.

PYTHON • type this in Colab

```
import numpy as np

def compute_reciprocals(values):
    output = np.empty(len(values))
    for i in range(len(values)):
        output[i] = 1.0 / values[i]
    return output

print(compute_reciprocals(np.array([2, 4, 5, 10])))
```

Q6 BUILD IT

Write code that measures, with `time.perf_counter()`, how long the loop version and the vectorized version `1.0 / big` take on 1,000,000 numbers, and print how many times faster the vectorized one is.

Q7 THEORY

[4 marks] In your own words, what does "pushing the loop into the compiled layer" mean, and why does it remove the slowness?

PART 2

2 Introducing Ufuncs

Block 3: vectorized operations and universal functions

IN THIS PART YOU WILL LEARN

- ✓ What a ufunc and a vectorized operation are
- ✓ How `1.0 / values` replaces a whole loop and gives the same answer
- ✓ The three input shapes: scalar with array, array with array, multi-dimensional
- ✓ Unary versus binary ufuncs, and the golden rule: no loops over arrays

2.1 The cure: vectorized operations

A NumPy array is **homogeneous**: every element has the same type. NumPy therefore checks the type **once** and hands the work to a statically typed, pre-compiled routine. This is called a **vectorized operation**. The loop still happens, but it has been pushed down from slow Python into fast compiled code.

For simple operations such as element-wise division, vectorization is as easy as using the normal Python operator directly on the array. The functions that do the work behind the scenes are called **universal functions**, or **ufuncs**. Their main purpose is to quickly run the same operation on every value of a NumPy array.

INTERVIEW FLAG

"What is a ufunc? What is vectorization?"

Ufunc = a NumPy function that applies an operation to every element of an array using fast compiled code. **Vectorization** = replacing a Python loop with such an operation, so the loop runs in compiled code.

2.2 Same answer, a thousand times faster

PYTHON • type this in Colab

```
import numpy as np
rng = np.random.default_rng(seed=1701)
values = rng.integers(1, 10, size=5)

def compute_reciprocals(values):
    output = np.empty(len(values))
    for i in range(len(values)):
        output[i] = 1.0 / values[i]
    return output

print(compute_reciprocals(values))   # the loop
print(1.0 / values)                  # the ufunc
```

▶ OUTPUT

```
[0.11111111 0.25       1.         0.33333333 0.125     ]
[0.11111111 0.25       1.         0.33333333 0.125     ]
```

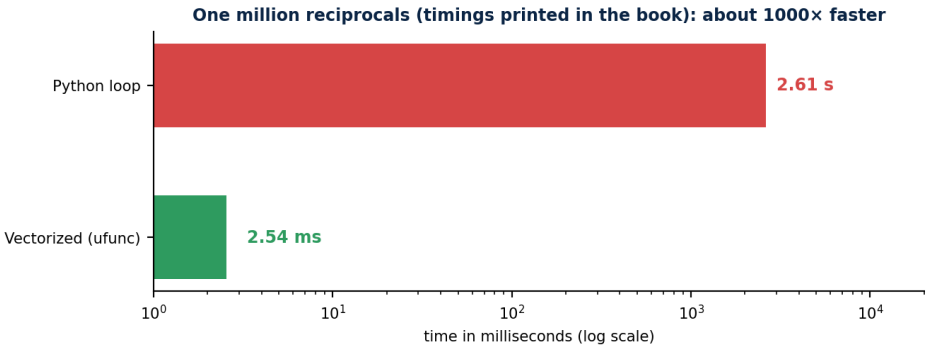
Identical results, but the second is one expression: no loop, no empty `output` array to prepare. Now the clock, on one million numbers:

PYTHON • type this in Colab

```
%timeit (1.0 / big_array)
```

▶ OUTPUT

```
2.54 ms ± 383 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)
```



Timings printed in the book (log scale, so the huge gap fits on the page).

Method	Time for 1 million reciprocals
Python loop	2.61 **seconds**
Vectorized ufunc	2.54 **milliseconds**

Seconds against milliseconds is roughly **1000 times faster**, and the gap grows as the arrays grow.

TRY IT

Measure it yourself in Colab and compare the units on each line:

`%timeit [1.0 / x for x in range(1, 1_000_001)]` versus `%timeit 1.0 / np.arange(1, 1_000_001)`

The exact numbers depend on your machine, so none are printed here.

2.3 Ufuncs are flexible: three kinds of input

Scalar and array

A single number combined with an array is applied to **every** element. This is what `1.0 / values` did.

Two arrays

Two arrays of the same shape work **position by position**: first with first, second with second, and so on.

PYTHON • type this in Colab

```
print(np.arange(5))
print(np.arange(1, 6))
print(np.arange(5) / np.arange(1, 6))
```

▶ OUTPUT

```
[0 1 2 3 4]
[1 2 3 4 5]
[0.         0.5        0.66666667 0.75       0.8       ]
```

Read it as 0/1, 1/2, 2/3, 3/4, 4/5.

Multi-dimensional arrays

Ufuncs are not limited to 1D. They act on every element of a 2D or 3D array in the same way, and no nested loops are needed.

PYTHON • type this in Colab

```
x = np.arange(9).reshape((3, 3))
print(x)
print(2 ** x)
```

▶ OUTPUT

```
[[0 1 2]
 [3 4 5]
 [6 7 8]]
[[  1   2   4]
 [  8  16  32]
 [ 64 128 256]]
```

Every element became 2 raised to that element: $2^0 = 1$, $2^1 = 2$, $2^2 = 4$ and so on.

COMMON TRAP

Two arrays must match in shape. Position-by-position only works when each position has a partner. (NumPy can sometimes stretch a smaller array to fit; that is called *broadcasting* and it has its own chapter next.)

PYTHON • type this in Colab

```
np.arange(3) + np.arange(4)
```

▶ OUTPUT

```
ValueError: operands could not be broadcast together with shapes (3,) (4,)
```

2.4 Unary and binary ufuncs

Ufuncs come in two flavours, depending on how many inputs they take.

Kind	Inputs	Example	In words
Unary	one array	`np.negative(x), np.abs(x)`	changes each element on its own
Binary	two inputs	`np.add(x, y), np.multiply(x, 2)`	combines two things element by element

2.5 The golden rule

INTERVIEW POINT

Computation using vectorization through ufuncs is nearly always more efficient than the equivalent Python loop, especially as the arrays grow. **Any time you see a loop over a NumPy array, ask: can this be replaced by a vectorized expression?**

WHY IT MATTERS

PyTorch and scikit-learn copy this same idea. When you later write a loss function or normalise a dataset, the code you write will be one or two vectorized lines, not a loop. Getting the habit now saves hours of training time later.

✎ **PRACTICE PROBLEMS | Part 2: Introducing Ufuncs**

Q1 THEORY

[3 marks] Define **ufunc** and **vectorization**, and explain in two lines why a vectorized operation is faster than a loop.

Q2 PREDICT THE OUTPUT

Predict the output.

PYTHON • type this in Colab

```
import numpy as np
print(1.0 / np.array([1, 2, 4, 5]))
```

Q3 BUILD IT

Rewrite this slow code as **one vectorized line**, then predict its output by hand.

PYTHON • type this in Colab

```
import numpy as np
a = np.array([1, 2, 3, 4, 5])

result = np.empty(len(a))
for i in range(len(a)):
    result[i] = (a[i] ** 2) + 10
```

Q4 PREDICT THE OUTPUT

Predict the output of the two-array operation.

PYTHON • type this in Colab

```
import numpy as np
print(np.arange(4) * np.arange(10, 14))
```

Q5 PREDICT THE OUTPUT

Predict the 2D result.

PYTHON • type this in Colab

```
import numpy as np
x = np.arange(6).reshape(2, 3)
print(3 ** x)
```

Q6 FIND THE BUGS

This line crashes. Explain the error and say what kind of mistake it is.

PYTHON • type this in Colab

```
import numpy as np
print(np.arange(3) + np.arange(4))
```

Q7 THINK IT THROUGH

You read a script containing `for i in range(len(a)): b[i] = a[i] * 0.5 + 1`. What do you do, and why?

Q8 READ & DECODE

Explain in plain words what `2 ** x` does when `x` is a 3×3 array. Which operator and which ufunc are involved?

PART 3

3 Array Arithmetic

Block 3: the operators that are secretly ufuncs

IN THIS PART YOU WILL LEARN

- ✓ The eight arithmetic operators and the ufunc behind each one
- ✓ Order of operations and how to chain expressions
- ✓ Result types: when you get ints and when you get floats
- ✓ Floor division and remainder, including the negative-number trap

3.1 The basic operators

NumPy ufuncs feel natural because they use **Python's own arithmetic operators**. Addition, subtraction, multiplication, division and floor division all work directly on arrays, element by element.

PYTHON • type this in Colab

```
x = np.arange(4)
print("x      =", x)
print("x + 5  =", x + 5)
print("x - 5  =", x - 5)
print("x * 2  =", x * 2)
print("x / 2  =", x / 2)
print("x // 2 =", x // 2)   # floor division
```

▶ OUTPUT

```
x      = [0 1 2 3]
x + 5  = [5 6 7 8]
x - 5  = [-5 -4 -3 -2]
x * 2  = [0 2 4 6]
x / 2  = [0.  0.5 1.  1.5]
x // 2 = [0 0 1 1]
```

There is also a unary negation (`-x`), `**` for powers and `%` for the remainder:

PYTHON • type this in Colab

```
print("-x     = ", -x)
print("x ** 2 = ", x ** 2)
print("x % 2  = ", x % 2)
```

▶ OUTPUT

```
-x     =  [ 0 -1 -2 -3]
x ** 2 =  [0 1 4 9]
x % 2  =  [0 1 0 1]
```

3.2 Chaining and order of operations

You can string operators together however you like; the usual order of operations (brackets, then powers, then multiplication and division, then addition and subtraction) is respected.

PYTHON • type this in Colab

```
print(-(0.5 * x + 1) ** 2)
```

▶ OUTPUT

```
[-1.   -2.25 -4.   -6.25]
```

Trace it step by step for `x = [0 1 2 3]`:

Step	Expression	Result
1	`0.5 * x`	[0. 0.5 1. 1.5]
2	`0.5 * x + 1`	[1. 1.5 2. 2.5]
3	`(0.5 * x + 1) ** 2`	[1. 2.25 4. 6.25]
4	`-(...)` flips the sign last	[-1. -2.25 -4. -6.25]

COMMON TRAP

`-(0.5*x + 1) ** 2` is **not** the square of a negative number. `**` binds tighter than the leading minus, so the power is done first and the sign is flipped at the end. That is why every answer is negative.

3.3 The hidden truth: operators are wrappers

Every operator is just a shortcut for a named ufunc stored inside NumPy. Writing `x + 2` secretly calls `np.add(x, 2)`.

PYTHON • type this in Colab

```
print(x + 2)
print(np.add(x, 2))
```

▶ OUTPUT

```
[2 3 4 5]
[2 3 4 5]
```

Operator	Equivalent ufunc	Description
`+`	`np.add`	Addition (1 + 1 = 2)
`-`	`np.subtract`	Subtraction (3 - 2 = 1)
`- (unary)`	`np.negative`	Negation (-2)
`*`	`np.multiply`	Multiplication (2 * 3 = 6)
`/`	`np.divide`	Division (3 / 2 = 1.5)
`//`	`np.floor_divide`	Floor division (3 // 2 = 1)
`**`	`np.power`	Exponentiation (2 ** 3 = 8)
`%`	`np.mod`	Modulus / remainder (9 % 4 = 1)

MEMORY TRICK

You will type the **operators** 99% of the time. The ufunc names in this table are only for **recognising** them when you read other people's code or an error message. Do not memorise the table.

3.4 What type do you get back?

The result type follows simple rules, and getting them wrong causes silent bugs (you met the same idea in the data-types notes).

PYTHON • type this in Colab

```
x = np.arange(4)
print((x / 2).dtype)      # true division
print((x // 2).dtype)     # floor division
print((x * 2).dtype)      # int times int
print((x * 2.0).dtype)    # int times float
print(np.array([4, 6]) / 2)
```

▶ OUTPUT

```
float64
int64
int64
float64
[2. 3.]
```

Operation	Result type	Why
/	always float	division may not come out whole, so NumPy returns decimals even for 4 / 2 = 2.0
// % + - * on ints	int	whole numbers stay whole
anything with a float	float	upcasting: int goes up to float

3.5 Floor division and remainder: the traps

// is **floor** division: it rounds the true answer down towards **negative infinity**, not towards zero. % always takes the sign of the divisor (the number on the right).

PYTHON • type this in Colab

```
print(7 // 2, -7 // 2)
print(7 % 3, -7 % 3)
print(np.array([-7, 7]) // 2)
print(np.array([-7, 7]) % 2)
```

▶ OUTPUT

```
3 -4
1 2
[-4  3]
[1 1]
```

COMMON TRAP

-7 // 2 is **-4**, not -3. Floor means "round down on the number line", and the number just below -3.5 is -4. Many people get this wrong in vivas and in code.

Why bother? In ML, // and % are used to organise data into batches. Which batch does sample number i fall in, and what is its position inside that batch?

PYTHON • type this in Colab

```
batch_size = 4
i = np.arange(10)
print("batch   :", i // batch_size)
print("position:", i % batch_size)
```

▶ OUTPUT

```
batch   : [0 0 0 0 1 1 1 1 2 2]
position: [0 1 2 3 0 1 2 3 0 1]
```

IMPORTANT

Dividing by zero: NumPy does not crash. Float division gives inf, -inf or nan and prints a warning. Silent inf or nan values spreading through a model is one of the most common ways training breaks.

PYTHON • type this in Colab

```
print(np.array([1.0, -1.0, 0.0]) / 0)
```

▶ OUTPUT

```
[ inf -inf  nan]
```

✎ PRACTICE PROBLEMS | Part 3: Array Arithmetic

Q1 PREDICT THE OUTPUT

Let `x = np.array([10, 15, 20, 25])`. Predict the three results.

PYTHON • type this in Colab

```
import numpy as np
x = np.array([10, 15, 20, 25])
print(x // 4)
print(x % 4)
print(-(x / 5) + 1)
```

Q2 READ & DECODE

Name the ufunc behind each: (a) `x * 2` (b) `-x` (c) `x // 2` (d) `x % 2` (e) `x ** 2`.

Q3 PREDICT THE OUTPUT

Predict the output. Watch the order of operations.

PYTHON • type this in Colab

```
import numpy as np
x = np.array([1, 2, 3])
print(x ** 2 + 3 * x - 1)
```

Q4 CALCULATE

Work these out by hand, then check: (a) `7 // 2` (b) `-7 // 2` (c) `7 % 3` (d) `-7 % 3`.

Q5 PREDICT THE OUTPUT

Predict the three dtypes.

PYTHON • type this in Colab

```
import numpy as np
print((np.arange(3) / 1).dtype)
print((np.arange(3) // 1).dtype)
print((np.arange(3) + 0.5).dtype)
```

Q6 BUILD IT

There are 10 samples (numbered 0 to 9) and each batch holds 4. Using ufuncs only, compute the batch number and the position inside the batch for every sample.

Q7 FIND THE BUGS

A student wants integer halves `[0 1 1]` of `[1, 2, 3]` but gets decimals. Why, and what is the fix?

PYTHON • type this in Colab

```
import numpy as np
print(np.array([1, 2, 3]) / 2)
```

Q8 THINK IT THROUGH

What is the difference between `x + 2` and `np.add(x, 2)`? Which would you write in real code?

Q9 BUILD IT

Convert the temperatures `[0, 37, 100]` degrees Celsius to Fahrenheit in one vectorized line (F = C × 9/5 + 32).

PART 4

4 Absolute Value

Block 3: np.abs, complex magnitude and the Mean Absolute Error

IN THIS PART YOU WILL LEARN

- ✓ The three names for absolute value, and which one to type
- ✓ What absolute value means for complex numbers (magnitude)
- ✓ Mean Absolute Error step by step: why abs is used in ML
- ✓ A silent trap: the absolute value of the most negative integer

4.1 Three names, one ufunc

NumPy understands Python's built-in `abs()` function, and it has its own ufunc as well. All three do exactly the same thing: they turn every negative number into a positive one and leave positives alone.

PYTHON • type this in Colab

```
x = np.array([-2, -1, 0, 1, 2])
print(abs(x))
print(np.absolute(x))
print(np.abs(x))
```

▶ OUTPUT

```
[2 1 0 1 2]
[2 1 0 1 2]
[2 1 0 1 2]
```

Name	What it is	Use it?
`abs(x)`	Python built-in that hands the work to NumPy	works
`np.absolute(x)`	the real ufunc, long name	works
`np.abs(x)`	short alias of `np.absolute`	**this one, in practice**

4.2 Complex numbers: absolute value is the magnitude

A complex number such as `3 - 4j` has a real part and an imaginary part. Its absolute value is its **magnitude**: the distance from the origin, found with the Pythagorean theorem. For `3 - 4j` that is $\sqrt{(3^2 + 4^2)} = \sqrt{25} = 5$.

PYTHON • type this in Colab

```
x = np.array([3 - 4j, 4 - 3j, 2 + 0j, 0 + 1j])
print(np.abs(x))
```

▶ OUTPUT

```
[5. 5. 2. 1.]
```

NOTE

This is the same idea as the **length of a vector** from your Linear Algebra videos. Complex numbers are rare in NLP and GenAI, so recognise the idea but do not spend time on it.

4.3 Why abs matters in ML: Mean Absolute Error

A model makes predictions (`y_pred`) and we compare them with the true answers (`y_true`). The **error** of each sample is `y_true - y_pred`. It can be positive (predicted too low) or negative (predicted too high).

We want **one number** that says how wrong the model is overall. The obvious idea, averaging the raw errors, **fails**, because positive and negative errors cancel:

PYTHON • type this in Colab

```
errors = np.array([4, -3, 0, -7])
print(np.mean(errors))          # raw average
```

▶ OUTPUT

```
-1.5
```

The model is clearly wrong by several units, yet the raw average looks small. And for errors `[+5, -5]` it would be exactly 0, a "perfect" score for a bad model. The fix has two steps:

Step	Code	Result for [4, -3, 0, -7]	What it does
1	`np.abs(errors)`	[4 3 0 7]	removes the signs: every error now counts as "how far off"
2	`np.mean(...)`	(4 + 3 + 0 + 7) / 4 = 3.5	adds them up and divides by how many

PYTHON • type this in Colab

```
mae = np.mean(np.abs(errors))
print(mae)
```

▶ OUTPUT

```
3.5
```

This is the **Mean Absolute Error (MAE)**. In words: "on average, the model is off by 3.5 units." `np.mean` is an *aggregation* (many values in, one value out); aggregations get their own chapter (Part 7 onwards), so for now read it as "the average of the array".

WHY IT MATTERS

MAE is one of the first loss functions you will meet. The same `np.abs` also gives **L1 regularisation**, where you add up the absolute values of a model's weights: `np.sum(np.abs(weights))`.

4.4 A silent trap: the most negative integer

Integer types have a fixed range (you saw this in the data-types notes). An `int8` holds -128 to 127. There is no +128, so what happens to `abs(-128)`?

PYTHON • type this in Colab

```
a = np.array([-128], dtype=np.int8)
print(np.abs(a))
```

▶ OUTPUT

```
[-128]
```

COMMON TRAP

The answer comes back negative. +128 does not fit in int8, so it wraps around to -128 and NumPy raises no error. This is the same silent overflow you met with `uint8`. If your data is small integer types, convert with `astype` to a larger type first.

✎ PRACTICE PROBLEMS | Part 4: Absolute Value

Q1 PREDICT THE OUTPUT

Predict the output.

PYTHON • type this in Colab

```python
import numpy as np
print(np.abs(np.array([-5, 3, 0, -1.5])))
```

Q2 CALCULATE

Find the magnitude of each complex number by hand, then verify: `5 + 12j` and `6 - 8j`. Show the working.

Q3 BUILD IT

Given `y_true = [3, 5, 2, 7]` and `y_pred = [2, 5, 4, 10]`, compute the Mean Absolute Error in NumPy.

Q4 THINK IT THROUGH

Why can we not simply average the raw errors `[+5, -5]` to judge a model? What does the abs step change?

Q5 FIND THE BUGS

Both lines crash. Explain each error message and fix them.

PYTHON • type this in Colab

```python
import numpy as np
errors = np.array([4, -3, 0, -7])
print(np.abs[errors])
print(errors.abs())
```

Q6 PREDICT THE OUTPUT

Predict the result, then explain it.

PYTHON • type this in Colab

```python
import numpy as np
print(np.abs(np.array([-128], dtype=np.int8)))
```

Q7 THEORY

[3 marks] Define MAE, write its one-line NumPy expression, and name one other place in ML where `np.abs` is used.

PART 5

5 Trigonometric Functions

Block 3: sin, cos, tan, their inverses and floating-point precision

IN THIS PART YOU WILL LEARN

- ✓ Radians, `np.pi` and how `np.linspace` makes a list of angles
- ✓ `np.sin`, `np.cos`, `np.tan` and the inverse functions
- ✓ Why `sin(π)` is not exactly 0, and how to compare floats safely
- ✓ Converting between degrees and radians
- ✓ Where trigonometry shows up in AI/ML

5.1 Angles in NumPy are radians

NumPy's trig functions expect angles in **radians**, not degrees. A half-turn is π radians (180 degrees), a quarter-turn is π/2 (90 degrees).

Degrees	Radians	In NumPy
0	0	`0`
90	π / 2	`np.pi / 2`
180	π	`np.pi`
360	2π	`2 * np.pi`

To try the functions we need some angles. `np.linspace(start, stop, num)` gives exactly `num` evenly spaced points with **both ends included**:

PYTHON • type this in Colab

```
theta = np.linspace(0, np.pi, 3)
print(theta)
```

▶ OUTPUT

```
[0.         1.57079633 3.14159265]
```

COMMON TRAP

`linspace` counts **points**, not gaps. 3 points from 0 to π means 2 gaps of π/2. `np.linspace(0, 10, 5)` has 4 gaps, so the step is 10 / 4 = 2.5, not 10 / 5.

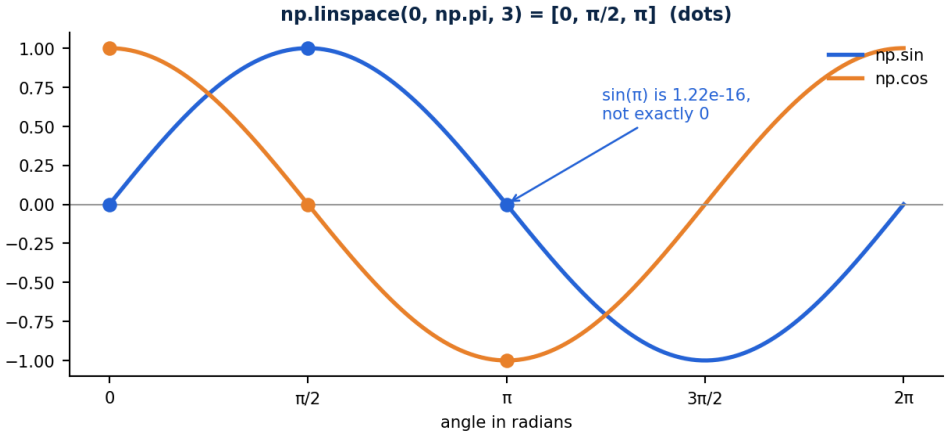
5.2 sin, cos and tan

PYTHON • type this in Colab

```
print("theta      = ", theta)
print("sin(theta) = ", np.sin(theta))
print("cos(theta) = ", np.cos(theta))
print("tan(theta) = ", np.tan(theta))
```

▶ OUTPUT

```
theta      =  [0.         1.57079633 3.14159265]
sin(theta) =  [0.0000000e+00 1.0000000e+00 1.2246468e-16]
cos(theta) =  [ 1.000000e+00  6.123234e-17 -1.000000e+00]
tan(theta) =  [ 0.00000000e+00  1.63312394e+16 -1.22464680e-16]
```



sin and cos over one full turn. The dots are the three angles above.

Read the dots: at 0 the sine is 0 and the cosine is 1; at π/2 the sine is 1 and the cosine is 0; at π the sine is 0 and the cosine is -1.

5.3 The surprise: why not exactly zero?

Look again at the output. `sin(π)` should be exactly 0, but NumPy gives `1.2246468e-16`; `cos(π/2)` gives `6.1e-17`; `tan(π/2)` gives a huge number instead of "undefined". The numbers are right to within **machine precision**.

Reason: a computer cannot store π exactly. `np.pi` is a very close approximation, so `sin` of that slightly-off number is slightly off zero. It is the floating-point limitation again.

PYTHON • type this in Colab

```
print(np.sin(np.pi) == 0)
print(np.isclose(np.sin(np.pi), 0))
```

▶ OUTPUT

```
False
True
```

COMMON TRAP

Never compare floats with `==`. `np.sin(np.pi) == 0` is `False`. Use `np.isclose(a, b)`, which asks "are these equal to within a tiny tolerance?" This matters when you test whether a model's output matches an expected value.

5.4 The inverse functions

`np.arcsin`, `np.arccos` and `np.arctan` go the other way: you give a value, they return the **angle** (in radians) that produces it.

PYTHON • type this in Colab

```
x = [-1, 0, 1]
print("arcsin(x) =", np.arcsin(x))
print("arccos(x) =", np.arccos(x))
print("arctan(x) =", np.arctan(x))
```

▶ OUTPUT

```
arcsin(x) = [-1.57079633  0.          1.57079633]
arccos(x) = [3.14159265 1.57079633 0.        ]
arctan(x) = [-0.78539816  0.          0.78539816]
```

PYTHON • type this in Colab

```
print(np.arcsin(2))
```

▶ OUTPUT

```
nan
```

NOTE

`arcsin` and `arccos` only accept values between -1 and 1 (a sine can never be 2). Outside that range NumPy returns `nan` (and prints a warning). Inverse trig is much less common in ML than sin and cos; just know it exists.

5.5 Degrees and radians

PYTHON • type this in Colab

```
print(np.deg2rad([0, 90, 180, 360]))
print(np.rad2deg(np.pi))
print(np.sin(90))                  # wrong: 90 is read as 90 radians
print(np.sin(np.deg2rad(90)))      # right
```

▶ OUTPUT

```
[0.         1.57079633 3.14159265 6.28318531]
180.0
0.8939966636005579
1.0
```

COMMON TRAP

`np.sin(90)` does **not** give 1. NumPy reads 90 as radians. Convert first: `np.sin(np.deg2rad(90))`. `np.radians` and `np.degrees` are aliases.

5.6 Where you meet trig in AI/ML

WHY IT MATTERS

Transformers (and so ChatGPT-style models): the original positional encoding, which tells a model the order of words, is built from sine and cosine waves of different frequencies. You will meet it in the NLP phase.

Cosine similarity: RAG and semantic search compare two embeddings by the angle between them. You compute it with a dot product, but the idea is the cosine of an angle.

Floating-point behaviour: the 1.22e-16 lesson returns every time you debug a numeric problem.

✎ **PRACTICE PROBLEMS | Part 5: Trigonometric Functions**

Q1 PREDICT THE OUTPUT

Predict the three printed values.

PYTHON • type this in Colab

```
import numpy as np
print(np.sin(np.pi / 2))
print(np.cos(0))
print(np.isclose(np.sin(np.pi), 0))
```

Q2 THINK IT THROUGH

Why does `np.sin(np.pi)` print `1.2246468e-16` instead of exactly 0? Answer in your own words.

Q3 PREDICT THE OUTPUT

Predict all five values.

PYTHON • type this in Colab

```
import numpy as np
print(np.linspace(0, 10, 5))
```

Q4 CALCULATE

Convert 0, 90, 180 and 360 degrees to radians using NumPy, and 3.14159 radians back to degrees (approximately).

Q5 **FIND THE BUGS**

A student expects `np.sin(90)` to return 1 but gets another number. Explain and fix.

PYTHON • type this in Colab

```
import numpy as np
print(np.sin(90))
```

Q6 **PREDICT THE OUTPUT**

Predict the output of the inverse functions.

PYTHON • type this in Colab

```
import numpy as np
print(np.arccos([1, 0, -1]))
```

Q7 **THEORY**

[3 marks] Why should floating-point numbers never be compared with `==`? What do you use instead? Give an example.

PART 6

6 Exponents and Logarithms

Block 3: the most important ufuncs in this chapter for AI/ML

IN THIS PART YOU WILL LEARN

- ✓ `np.exp`, `np.exp2`, `np.power` and what they compute
- ✓ Logarithms as the inverse of exponentials, and the `np.log` trap
- ✓ Domain rules: log of 0 and of negative numbers
- ✓ `np.expm1` and `np.log1p` for tiny numbers
- ✓ Sigmoid, softmax and cross-entropy: the payoff in ML

6.1 Exponentials

PYTHON • type this in Colab

```
x = [1, 2, 3]
print("x   =", x)
print("e^x =", np.exp(x))
print("2^x =", np.exp2(x))
print("3^x =", np.power(3., x))
```

▶ OUTPUT

```
x   = [1, 2, 3]
e^x = [ 2.71828183  7.3890561  20.08553692]
2^x = [2. 4. 8.]
3^x = [ 3.  9. 27.]
```

Function	Computes	Notes	Priority
`np.exp(x)`	e^x	e is Euler's number, about 2.718: the "natural" exponential	MUST KNOW
`np.exp2(x)`	2^x	powers of two	LOOK UP
`np.power(b, x)`	b^x	any base; same as `b ** x`	GOOD TO KNOW

Here `x` is a plain Python list, not an array. Ufuncs convert lists automatically.

6.2 Logarithms

A logarithm is the **inverse** of an exponential. If $2^3 = 8$, then $\log_2(8) = 3$. A log answers: "*what power gives me this number?*"

PYTHON • type this in Colab

```
x = [1, 2, 4, 10]
print("x        =", x)
print("ln(x)    =", np.log(x))
print("log2(x)  =", np.log2(x))
print("log10(x) =", np.log10(x))
```

▶ OUTPUT

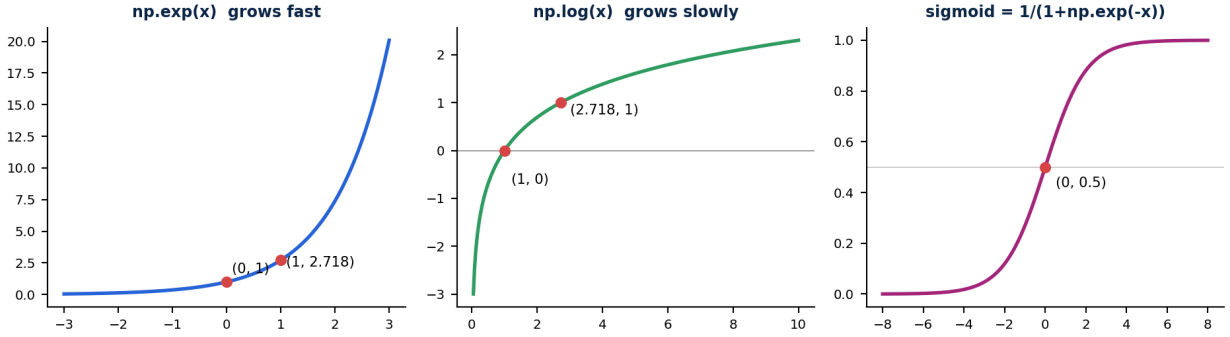
```
x        = [1, 2, 4, 10]
ln(x)    = [0.         0.69314718 1.38629436 2.30258509]
log2(x)  = [0.         1.         2.         3.32192809]
log10(x) = [0.         0.30103    0.60205999 1.        ]
```

Function	Base	Quick mental checks
`np.log(x)`	e (natural log, "ln")	np.log(1) = 0 because $e^0 = 1$

Function	Base	Quick mental checks
`np.log2(x)`	2	np.log2(8) = 3 because $2^3 = 8$
`np.log10(x)`	10	np.log10(1000) = 3 because $10^3 = 1000$

COMMON TRAP `np.log` **is the natural log (base e), NOT base 10.** Students used to school maths expect `log(100)` to be 2. `np.log(100)` is about 4.605. For base 10 write `np.log10`.

MEMORY TRICK **np.log = ln.** If you want any other base, the base is written in the name: `log2`, `log10`.



exp grows explosively, log grows very slowly, sigmoid squashes any number into 0 to 1.

6.3 Exp and log undo each other, and the domain rules

PYTHON • type this in Colab

```
x = np.array([1.0, 2.0, 3.0])
print(np.log(np.exp(x)))
print(np.exp(np.log(5)))
print(np.log(np.array([1, 0, -1])))
```

▶ OUTPUT

```
[1. 2. 3.]
4.999999999999999
[  0. -inf  nan]
```

The first line recovers the original numbers. The second gives 5 up to tiny rounding (4.999999999999999 on most machines: precision again). The third shows the **domain rules**: a log only accepts **positive** numbers. `log(0)` is `-inf` and `log` of a negative number is `nan`.

6.4 Precision tricks: expm1 and log1p

NumPy also has two special versions for very small inputs: `np.expm1(x)` computes `exp(x) - 1` and `np.log1p(x)` computes `log(1 + x)`. Why have separate functions?

PYTHON • type this in Colab

```
x = 1e-16
print(np.log(1 + x))    # 1 + 1e-16 is stored as exactly 1.0
print(np.log1p(x))      # keeps the tiny value
```

▶ OUTPUT

```
0.0
1e-16
```

When `x` is tiny, the computer loses it as soon as it forms `1 + x`. `log1p` and `expm1` avoid that step and keep the precision.

NOTE You will rarely type these yourself. You will see them inside library code that is written to be numerically stable. Priority: LOOK UP.

6.5 The payoff: exp and log inside ML

Sigmoid: one number squashed into 0 to 1

PYTHON • type this in Colab

```
x = np.array([-2.0, 0.0, 2.0])
sigmoid = 1 / (1 + np.exp(-x))
print(sigmoid)
```

▶ OUTPUT

```
[0.11920292 0.5        0.88079708]
```

Sigmoid is the activation function of logistic regression (coming up in CS229). Large positive inputs go near 1, large negative inputs near 0, and 0 gives exactly 0.5.

Softmax: scores into probabilities

PYTHON • type this in Colab

```
scores = np.array([2.0, 1.0, 0.1])
e = np.exp(scores)
probs = e / e.sum()
print(probs)
print(probs.sum())
```

▶ OUTPUT

```
[0.65900114 0.24243297 0.09856589]
1.0
```

`exp` makes every score positive, and dividing by the total makes them add up to 1. This is the final step of every classifier and every language model: GPT turns scores for each possible next word into probabilities with softmax.

COMMON TRAP `exp` **overflows for large inputs.** Naive softmax on scores like `[1000, 1001]` produces `nan`. The standard fix is to subtract the maximum first (the answer does not change).

PYTHON • type this in Colab

```
z = np.array([1000.0, 1001.0])
print(np.exp(z) / np.exp(z).sum())          # naive: breaks
e = np.exp(z - z.max())
print(e / e.sum())                           # stable
```

▶ OUTPUT

```
[nan nan]
[0.26894142 0.73105858]
```

Cross-entropy: the loss that trains language models

If the model gives the correct answer a probability `p`, the loss for that sample is `-np.log(p)`. A confident correct answer costs almost nothing; a confident wrong answer is punished hard.

PYTHON • type this in Colab

```
p = np.array([0.9, 0.5, 0.1, 0.01])
print(-np.log(p))
```

▶ OUTPUT

```
[0.10536052 0.69314718 2.30258509 4.60517019]
```

Probability given to the right answer	Loss = -log(p)	Meaning
0.9	0.105	confident and right: tiny loss
0.5	0.693	unsure
0.1	2.303	mostly wrong: big loss
0.01	4.605	confidently wrong: huge loss

WHY IT MATTERS Remember **sigmoid, softmax and cross-entropy** and you have most of the maths of a classifier or a language model built out of two ufuncs: `np.exp` and `np.log`.

✎ PRACTICE PROBLEMS | Part 6: Exponents and Logarithms

Q1 CALCULATE

Work these out by hand, then check: `np.log2(8)`, `np.log10(1000)`, `np.log(1)`, `np.exp(0)`.

Q2 THINK IT THROUGH

A student writes `np.log(100)` expecting 2. What does it return (roughly)? Why was the student wrong?

Q3 BUILD IT

Write the one-line NumPy expression for sigmoid and apply it to `[-2, 0, 2]`.

Q4 PREDICT THE OUTPUT

Predict the output.

PYTHON • type this in Colab

```
import numpy as np
print(np.exp2([1, 2, 3, 10]))
print(np.power(3.0, [0, 1, 2]))
```

Q5 PREDICT THE OUTPUT

Without running it, what does softmax give for `[2, 2, 2]`? Then check with code.

Q6 THINK IT THROUGH

A model assigns probability 0.5 to the correct class. Compute the loss `-np.log(p)`, and explain why a probability of 0.01 gives a much bigger loss.

Q7 FIND THE BUGS

This loss code returns `inf` for one sample. Why? How would you protect against it?

PYTHON • type this in Colab

```
import numpy as np
probs = np.array([0.7, 0.0, 0.3])
print(-np.log(probs))
```

Q8 CALCULATE

For `x = 1e-10` compare `np.log(1 + x)` with `np.log1p(x)`. Which is more accurate? (The true value is 9.9999999995e-11.)

Q9 THEORY

[4 marks] Explain a logarithm as the inverse of an exponential. State the difference between `np.log` and `np.log10`, and give two uses of exp or log in ML.

PART 7

7 Aggregations: sum, min and max

Block 4, first section: collapsing many numbers into one

IN THIS PART YOU WILL LEARN

- ✓ What an aggregation (reduction) is and why every ML pipeline uses them
- ✓ Python's built-in `sum`, `min`, `max` versus NumPy's versions
- ✓ How much faster NumPy is, and why
- ✓ Function style versus method style (`np.sum(x)` or `x.sum()`)
- ✓ The trap: `sum(x, 1)` and `np.sum(x, 1)` mean different things

7.1 What is an aggregation?

A first step in exploring any dataset is to compute **summary statistics**: numbers that describe a whole pile of data in a few values. The most common are the mean and the standard deviation, which tell you the "typical" value and how spread out the data is. Others are the sum, the minimum and maximum and the median.

An operation that takes **many values in and gives one value out** is called an **aggregation** (also called a **reduction**, because it reduces the array to a single number).

ANALOGY

The class result. Sixty students write an exam. Instead of reading sixty marks you ask for the average, the highest and the lowest. Sixty numbers collapse into three summary numbers. That is exactly what aggregations do to an array.

WHY IT MATTERS

A loss value is the average of the errors in a batch. Normalising data uses a mean and a standard deviation. Picking a model's predicted class uses an argmax. Aggregations are everywhere.

7.2 Summing the values in an array

Python has a built-in `sum`, and NumPy has `np.sum`. On the simplest input they give the same answer. We use a seeded generator so the numbers repeat on your screen.

PYTHON • type this in Colab

```
import numpy as np
rng = np.random.default_rng(42)
L = rng.random(100)

print(sum(L))
print(np.sum(L))
```

▶ OUTPUT

```
48.67184582657889
48.671845826578874
```

The last digits can differ slightly: the two methods add the numbers in a different order, and floating-point rounding depends on the order.

VERSION NOTE

In NumPy 2, when a scalar is the **last line** of a notebook cell it is shown with its type, like `np.float64(52.1)`. Older books and tutorials show a plain `52.1`. It is the same number. Wrap it in `print()` to see the plain form.

PYTHON • type this in Colab

```
np.sum(L)
```

▶ OUTPUT

```
np.float64(48.671845826578874)
```

7.3 The speed difference

NumPy executes the operation in compiled code, so it is dramatically faster than the built-in version on a big array.

PYTHON • type this in Colab

```
big_array = rng.random(1000000)
%timeit sum(big_array)
%timeit np.sum(big_array)
```

▶ OUTPUT

```
89.9 ms ± 233 µs per loop (mean ± std. dev. of 7 runs, 10 loops each)
521 µs ± 8.37 µs per loop (mean ± std. dev. of 7 runs, 1000 loops each)
```

Operation on 1 million numbers	Python built-in	NumPy	NumPy is faster by about
sum	89.9 ms	521 µs	170 times
min	72 ms	564 µs	130 times

Timings are the ones printed in the book; yours will differ. Why the gap? Exactly the idea from Part 1: the built-in `sum` loops through the array in Python, checking each element's type, while `np.sum` runs one compiled loop over a block of identical types.

7.4 Minimum and maximum

PYTHON • type this in Colab

```
print(min(big_array), max(big_array))          # slow: Python built-ins
print(np.min(big_array), np.max(big_array))      # fast: NumPy
```

▶ OUTPUT

```
1.2500323287589765e-07 0.9999997172035572
1.2500323287589765e-07 0.9999997172035572
```

Same answers again, and again NumPy is much faster (72 ms against 564 µs in the book). For `min`, `max`, `sum` and several other aggregates there is also a **shorter, method style**:

PYTHON • type this in Colab

```
print(big_array.min(), big_array.max(), big_array.sum())
```

▶ OUTPUT

```
1.2500323287589765e-07 0.9999997172035572 500031.87097114034
```

Style	Looks like	Use
Function	`np.sum(x)`	works everywhere
Method	`x.sum()`	shorter; very common in day-to-day ML code

INTERVIEW POINT Whenever possible, use the **NumPy** version of these aggregates when working on NumPy arrays, never the Python built-ins.

7.5 The trap: same name, different meaning

The built-in `sum` and `np.sum` are **not identical**. Their optional second argument means something different:

Call	Second argument means	Example
`sum(x, 1)`	the **starting value** of the total	`sum([1, 2, 3], 10)` is 16
`np.sum(x, 1)`	the **axis** (direction) to sum along	see Part 8

PYTHON • type this in Colab

```
print(sum([1, 2, 3], 10))
a = np.array([[1, 2], [3, 4]])
print(np.sum(a, 1))
print(sum(a, 1))
```

▶ OUTPUT

```
16
[3 7]
[5 7]
```

COMMON TRAP

Never use the built-in `sum` on a NumPy array. On a 2D array it quietly gives a different answer ([5 7] above, because it adds row by row starting from 1), while `np.sum(a, 1)` gives row totals. No error is raised in either case.

✎ PRACTICE PROBLEMS | Part 7: sum, min and max

Q1 PREDICT THE OUTPUT

What does the built-in `sum([1, 2, 3], 10)` return, and what does the 10 mean?

PYTHON • type this in Colab

```
print(sum([1, 2, 3], 10))
```

Q2 THINK IT THROUGH

You must total an array of 5 million numbers. Do you write `sum(arr)` or `arr.sum()`? Give the reason using the right term for what makes the built-in slow.

Q3 BUILD IT

For `x = np.array([4, 9, 1, 7])` write one line each for the minimum, maximum and sum, using the method style.

Q4 PREDICT THE OUTPUT

Predict both lines. They look similar but are not the same.

PYTHON • type this in Colab

```
import numpy as np
a = np.array([[1, 2], [3, 4]])
print(np.sum(a, 1))
print(sum(a, 1))
```

Q5 CALCULATE

From the book's timings, `sum(big_array)` took 89.9 ms and `np.sum(big_array)` took 521 µs. How many times faster is NumPy?

Q6 **FIND THE BUGS**

A student wants the total of `[4, 9, 1]` (which is 14) but gets 15. Find the bug and fix it.

PYTHON • type this in Colab

```
import numpy as np
a = np.array([4, 9, 1])
print(sum(a, 1))
```

Q7 **THEORY**

[3 marks] State two reasons why `np.sum` is faster than the built-in `sum` on arrays, and one way the two differ in meaning.

PART 8

8 Multidimensional Aggregates and axis

Block 4: the most important idea in this chapter

IN THIS PART YOU WILL LEARN

- ✓ What an aggregate does by default on a 2D array
- ✓ The `axis` argument, and the rule that names what disappears
- ✓ How to predict the shape of the result every time
- ✓ Per-feature and per-sample aggregates, the pattern behind normalisation
- ✓ Higher dimensions and `keepdims`

8.1 Default: aggregate everything

One common aggregation is along a **row or a column** of a table. Say you have a 2D array with 3 rows and 4 columns:

PYTHON • type this in Colab

```
M = np.array([[0, 3, 1, 2],
              [1, 9, 7, 0],
              [4, 8, 3, 7]])
print(M)
print(M.shape)
print(M.sum())
```

▶ OUTPUT

```
[[0 3 1 2]
 [1 9 7 0]
 [4 8 3 7]]
(3, 4)
45
```

With no extra argument, every aggregation runs over **all elements** and returns one number: 0+3+1+2 + 1+9+7+0 + 4+8+3+7 = 45.

8.2 The axis argument

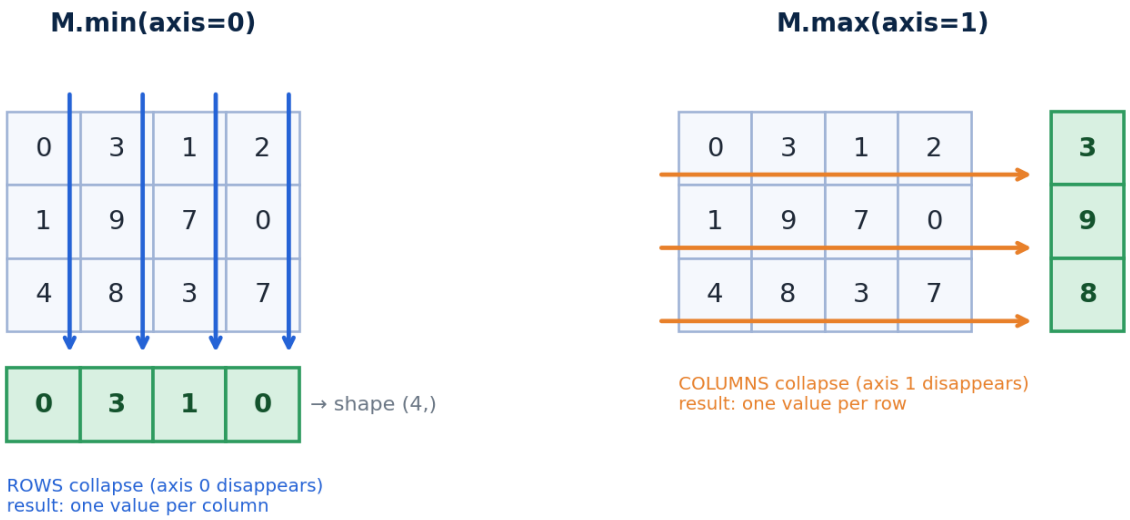
Aggregation functions accept an `axis` argument that says **in which direction to collapse**.

PYTHON • type this in Colab

```
print(M.min(axis=0))
print(M.max(axis=1))
print(M.sum(axis=0))
print(M.sum(axis=1))
```

▶ OUTPUT

```
[0 3 1 0]
[3 9 8]
[ 5 20 11  9]
[ 6 17 22]
```



axis=0 collapses downwards (one result per column). axis=1 collapses sideways (one result per row).

Call	Collapses	You get	Shape
`M.min(axis=0)`	the rows (goes down each column)	one value per **column**	(4,)
`M.max(axis=1)`	the columns (goes across each row)	one value per **row**	(3,)
`M.sum()`	everything	a single number	no shape

8.3 The confusing part, solved

The book calls this out: the way `axis` is specified confuses people. `axis=0` gives **column** results, yet the word "0" sounds like rows. The rule that never fails:

MEMORY TRICK

`axis` **names the dimension that DISAPPEARS, not the one you get back.** Equivalent shape trick: the axis you name is **removed from the shape**.

PYTHON • type this in Colab

```
print(M.shape)                  # (3, 4)
print(M.sum(axis=0).shape)      # axis 0 removed: (4,)
print(M.sum(axis=1).shape)      # axis 1 removed: (3,)
```

▶ OUTPUT

```
(3, 4)
(4,)
(3,)
```

INTERVIEW FLAG

"What does the axis argument mean in NumPy aggregations?"

Model answer: `axis` specifies the dimension that is collapsed (aggregated over), not the dimension that is returned. `axis=0` on a 2D array aggregates down the rows, giving one result per column.

IMPORTANT

Sanity check you can do on any result: the column totals and the row totals must both add up to the grand total. Here 5 + 20 + 11 + 9 = 45 and 6 + 17 + 22 = 45.

8.4 More than two dimensions

The same rule works for any number of dimensions: the named axis is deleted from the shape.

PYTHON • type this in Colab

```
A = np.zeros((3, 4, 5))
print(A.sum(axis=0).shape)
print(A.sum(axis=1).shape)
print(A.sum(axis=2).shape)
```

▶ OUTPUT

```
(4, 5)
(3, 5)
(3, 4)
```

NOTE

Shape `(32, 224, 224, 3)` is a batch of 32 colour images. `axis=0` would average **across the 32 images** and leave shape `(224, 224, 3)`. NumPy also accepts a tuple such as `axis=(1, 2)` to collapse several axes at once (LOOK UP).

8.5 keepdims: keep the collapsed axis as size 1

PYTHON • type this in Colab

```
print(M.sum(axis=1).shape)
print(M.sum(axis=1, keepdims=True).shape)
print(M.sum(axis=1, keepdims=True))
```

▶ OUTPUT

```
(3,)
(3, 1)
[[ 6]
 [17]
 [22]]
```

With `keepdims=True` the collapsed axis stays, with length 1, so the result still looks like a table. This matters later because it lets the result be stretched back across the original array (broadcasting, next chapter). Priority: GOOD TO KNOW.

8.6 The ML pattern: samples by features

ML data is almost always a 2D array of **samples × features**: each row is one example, each column one feature. Here four people, with height in cm and weight in kg:

PYTHON • type this in Colab

```
X = np.array([[170, 65],
              [160, 50],
              [180, 80],
              [175, 75]])
print(X.shape)
print(X.mean(axis=0))      # mean of each FEATURE (one per column)
print(X.max(axis=1))       # biggest value of each SAMPLE (one per row)
```

▶ OUTPUT

```
(4, 2)
[171.25  67.5 ]
[170 160 180 175]
```

Feature normalisation (standardisation) subtracts each feature's mean and divides by its standard deviation, so every feature is on a similar scale. It is one line:

PYTHON • type this in Colab

```
Z = (X - X.mean(axis=0)) / X.std(axis=0)
print(np.round(Z, 2))
print(np.round(Z.mean(axis=0), 2) + 0.0)
```

▶ OUTPUT

```
[[-0.17 -0.22]
 [-1.52 -1.53]
 [ 1.18  1.09]
 [ 0.51  0.65]]
[0. 0.]
```

Each feature (column) now has mean 0. NumPy stretched the one-row mean across all four rows automatically; that trick is **broadcasting**, the next chapter.

WHY IT MATTERS

`X.mean(axis=0)` and `X.std(axis=0)` for normalisation, and `probs.argmax(axis=1)` for predicted classes, are lines you will write in almost every project. PyTorch uses `dim=0` and `dim=1` for the same idea, so what you learn here carries over directly.

✎ PRACTICE PROBLEMS | Part 8: Multidimensional Aggregates and axis

Q1 PREDICT THE OUTPUT

For `A = [[2, 4, 6], [1, 3, 5]]` predict `A.sum(axis=0)`, `A.sum(axis=1)` and `A.sum()`, plus the shape of the first two.

PYTHON • type this in Colab

```
import numpy as np
A = np.array([[2, 4, 6],
              [1, 3, 5]])
print(A.sum(axis=0), A.sum(axis=0).shape)
print(A.sum(axis=1), A.sum(axis=1).shape)
print(A.sum())
```

Q2 CALCULATE

Give the shape of each result: (a) `(2, 3)` array with `axis=0` (b) `(2, 3)` with `axis=1` (c) `(1000, 20)` with `axis=0` (d) `(32, 64, 64, 3)` with `axis=0`.

Q3 BUILD IT

`X` has shape `(1000, 20)` (1000 samples, 20 features). Write the line that gives the mean of each feature, and state the shape of the result.

Q4 PREDICT THE OUTPUT

Predict `B.max(axis=1)` and `B.min(axis=0)`.

PYTHON • type this in Colab

```
import numpy as np
B = np.array([[5, 1, 9],
              [4, 8, 2],
              [7, 3, 6]])
print(B.max(axis=1))
print(B.min(axis=0))
```

Q5 FIND THE BUGS

A student wants the total of each **column** of `X` but writes `X.sum(axis=1)`. What do they get, and what is the fix?

PYTHON • type this in Colab

```
import numpy as np
X = np.array([[1, 2, 3],
              [4, 5, 6]])
print(X.sum(axis=1))
```

Q6 READ & DECODE

Explain in plain words: (a) `X.mean(axis=0)` (b) `X.max(axis=1)` (c) `X.sum()`, for `X` of shape (samples, features).

Q7 THINK IT THROUGH

`axis=0` is called the row axis, yet `M.sum(axis=0)` gives you column totals. Explain why this is not a contradiction.

Q8 BUILD IT

Standardise the 4×2 array `X` (height, weight) and verify each column of the result has mean (almost) 0 using `np.allclose`.

Q9 THEORY

[4 marks] Explain what `axis` means in an aggregation, give one example with a 2D array, and state the shape rule.

PART 9

9 The Core Aggregation Functions

Block 4: mean, median, var, std, argmin, argmax and the NaN-safe versions

IN THIS PART YOU WILL LEARN

- ✓ The aggregation functions you will use daily, with their priorities
- ✓ `mean` versus `median`, and why the median resists outliers
- ✓ Variance and standard deviation, worked by hand
- ✓ `argmax` and `argmin`: the index, not the value
- ✓ NaN-safe versions for data with missing values

9.1 The toolbox

NumPy has many aggregation functions, all with the same style (`np.name(x)` or `x.name()`, with an optional `axis`). Most also have a **NaN-safe** twin that ignores missing values. Here are the ones that matter now:

Function	NaN-safe version	Description	Priority
`np.sum`	`np.nansum`	total of the elements	MUST KNOW
`np.mean`	`np.nanmean`	average	MUST KNOW
`np.std`	`np.nanstd`	standard deviation (spread)	MUST KNOW
`np.var`	`np.nanvar`	variance (spread squared)	GOOD TO KNOW
`np.min / np.max`	`np.nanmin / np.nanmax`	smallest / largest value	MUST KNOW
`np.argmin / np.argmax`	`np.nanargmin / np.nanargmax`	**index** of the smallest / largest	MUST KNOW
`np.median`	`np.nanmedian`	middle value after sorting	GOOD TO KNOW

NOTE NumPy has a few more aggregations (`prod`, `percentile`, `any`, `all`). We leave them for later: you can look them up the day you need them.

For all the examples below we use one small array:

PYTHON • type this in Colab

```
x = np.array([2, 4, 4, 4, 5, 5, 7, 9])
```

9.2 mean and median

The **mean** is the sum divided by the count: (2+4+4+4+5+5+7+9) / 8 = 40 / 8 = 5. The **median** is the middle value after sorting; with 8 values it is the average of the two middle ones: (4 + 5) / 2 = 4.5.

PYTHON • type this in Colab

```
print(np.mean(x))
print(np.median(x))
```

▶ OUTPUT

```
5.0
4.5
```

Now make one value an **outlier** (a freak value). Change the 9 into 1000:

PYTHON • type this in Colab

```
y = np.array([2, 4, 4, 4, 5, 5, 7, 1000])
print(np.mean(y))
print(np.median(y))
```

▶ OUTPUT

```
128.875
4.5
```

INTERVIEW POINT The **mean** is dragged around by outliers; the **median** hardly moves. That is why the median is called **robust**. Always compare the two when you explore a dataset.

9.3 variance and standard deviation

Variance is the average of the squared distances from the mean. **Standard deviation** is its square root, which brings the spread back to the same units as the data.

Step	For x with mean 5	Result
1. distance from mean	each value minus 5	-3, -1, -1, -1, 0, 0, 2, 4
2. square them	removes the signs	9, 1, 1, 1, 0, 0, 4, 16 (sum 32)
3. average = variance	32 / 8	4.0
4. square root = std	√4	2.0

PYTHON • type this in Colab

```
print(np.var(x))
print(np.std(x))
```

▶ OUTPUT

```
4.0
2.0
```

COMMON TRAP `np.std` **divides by n (the population formula) by default.** Pandas and some other tools divide by n-1. Pass `ddof=1` to get that version. This is why two tools can print slightly different standard deviations for the same data.

PYTHON • type this in Colab

```
print(np.std(x, ddof=1))
```

▶ OUTPUT

```
2.138089935299395
```

9.4 argmax and argmin: the position, not the value

PYTHON • type this in Colab

```
print(np.max(x), np.argmax(x))
print(np.min(x), np.argmin(x))
print(np.argmax(np.array([3, 9, 9])))
```

▶ OUTPUT

```
9 7
2 0
1
```

COMMON TRAP `max` gives the **value**; `argmax` gives the **index** where it sits. Mixing them up is a very common bug. If the maximum appears more than once, `argmax` returns the **first** position.

Argmax with an axis picks the winner in each row. Imagine a classifier giving probabilities for 3 classes to 3 samples:

PYTHON • type this in Colab

```
probs = np.array([[0.1, 0.7, 0.2],
                  [0.8, 0.1, 0.1],
                  [0.2, 0.3, 0.5]])
print(probs.argmax(axis=1))
print(probs.max(axis=1))
```

▶ OUTPUT

```
[1 0 2]
[0.7 0.8 0.5]
```

WHY IT MATTERS

`argmax` **is how a model turns probabilities into a decision.** The predicted class of each sample is the position of its biggest probability. Language models do the same when they pick the most likely next token (greedy decoding).

9.5 NaN-safe versions

Real data has missing values, stored as `np.nan` ("not a number"). One NaN poisons an ordinary aggregate:

PYTHON • type this in Colab

```
d = np.array([1, np.nan, 3])
print(np.mean(d))
print(np.nanmean(d))
print(np.nansum(d), np.nanmax(d))
```

▶ OUTPUT

```
nan
2.0
4.0 3.0
```

MEMORY TRICK

Put `nan` **in front of the name** and the function ignores the missing values: `nanmean`, `nansum`, `nanstd`, `nanmax`, and so on. You do not need to memorise the list.

✎ PRACTICE PROBLEMS | Part 9: The Core Aggregation Functions

Q1 CALCULATE

For `y = [7, 3, 9, 3, 5]` work out by hand: the mean, the median (sort first), `argmax` and `argmin`.

Q2 CALCULATE

Work out the variance and standard deviation of `[1, 3, 5, 7]` by hand, then check.

Q3 THINK IT THROUGH

For `[2, 4, 4, 4, 5, 5, 7, 1000]` the mean is 128.9 but the median is 4.5. Which describes a "typical" value better, and why?

Q4 BUILD IT

A classifier outputs the probabilities below. Write one line giving the predicted class of every sample.

PYTHON • type this in Colab

```
import numpy as np
probs = np.array([[0.2, 0.5, 0.3],
                  [0.6, 0.3, 0.1],
                  [0.1, 0.1, 0.8]])
```

Q5 PREDICT THE OUTPUT

Predict the four outputs.

PYTHON • type this in Colab

```python
import numpy as np
d = np.array([4, np.nan, 2])
print(np.mean(d))
print(np.nanmean(d))
print(np.nansum(d))
print(np.nanmax(d))
```

Q6 FIND THE BUGS

A student wants the predicted class of each sample but gets probabilities instead. Find the bug and fix it.

PYTHON • type this in Colab

```python
import numpy as np
probs = np.array([[0.1, 0.9], [0.7, 0.3]])
pred = np.max(probs, axis=1)
print(pred)
```

Q7 THINK IT THROUGH

What does `ddof=1` change in `np.std`? Why can two libraries print different standard deviations for the same data?

Q8 READ & DECODE

Explain in plain words: (a) `np.nanstd(X, axis=0)` (b) `X.argmax(axis=1)` (c) `np.var(x)`.

Q9 THEORY

[4 marks] Explain the difference between `max` and `argmax` with an example, and give two uses of `argmax` in ML.

PART 10

10 Worked Example: Heights of US Presidents

Block 4: aggregations applied to a real dataset

IN THIS PART YOU WILL LEARN

- ✓ How to apply `mean`, `std`, `min`, `max`, `median` to real data
- ✓ What each summary number tells you
- ✓ How to read a histogram: bins, bars and the axes
- ✓ The mean-versus-median clue about the shape of the data

10.1 The data

The book summarises the heights, in centimetres, of all US presidents. The data lives in a CSV file: a text file of comma-separated values.

president_heights.csv (first lines)

```
order,name,height(cm)
1,George Washington,189
2,John Adams,170
3,Thomas Jefferson,189
```

The book loads it with pandas (covered in a later phase): `pd.read_csv(...)` reads the table and `np.array(data["height(cm)"])` pulls the height column out. After that it is an ordinary NumPy array. To keep things self-contained we type the numbers in directly:

PYTHON • type this in Colab

```
heights = np.array([189, 170, 189, 163, 183, 171, 185, 168, 173, 183, 173, 173,
                    175, 178, 183, 193, 178, 173, 174, 183, 183, 168, 170, 178,
                    182, 180, 183, 178, 182, 188, 175, 179, 183, 193, 182, 183,
                    177, 185, 188, 188, 182, 185, 191, 182])
print(heights.shape)
```

▶ OUTPUT

```
(44,)
```

VERSION NOTE Older printings of the book have 42 values and show a mean of 179.74. The current edition has 44 values and a mean of 180.05. The method is identical; only the dataset grew.

10.2 Summary statistics

PYTHON • type this in Colab

```
print("Mean height:       ", heights.mean())
print("Standard deviation:", heights.std())
print("Minimum height:    ", heights.min())
print("Maximum height:    ", heights.max())
```

▶ OUTPUT

```
Mean height:        180.04545454545453
Standard deviation: 6.983599441335736
Minimum height:     163
Maximum height:     193
```

Each call reduced the entire array of 44 numbers to **one** summarising value, which tells us about the distribution.

Statistic	Value	What it tells you
mean	180.05 cm	the typical height

Statistic	Value	What it tells you
std	6.98 cm	most presidents lie within roughly 7 cm of the mean
min / max	163 / 193 cm	the full range is 30 cm

PYTHON • type this in Colab

```
print(np.median(heights))
```

▶ OUTPUT

```
182.0
```

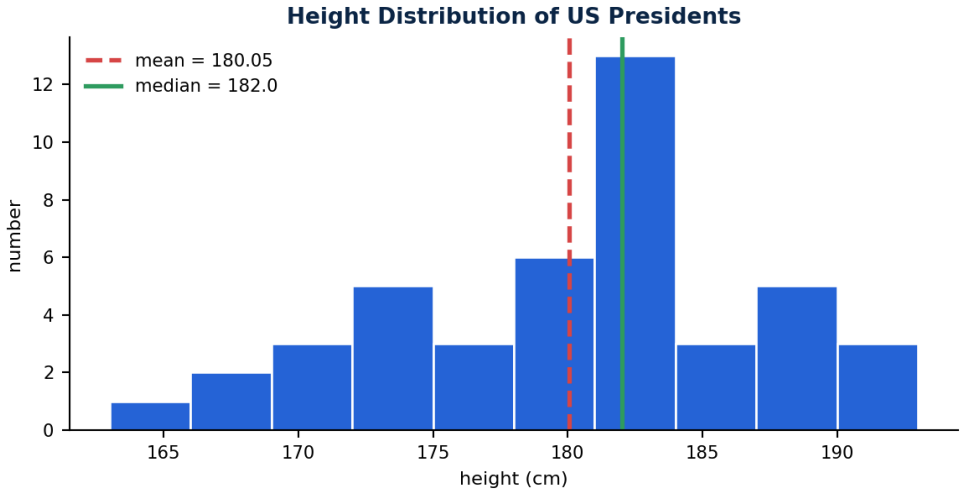
The median is 182 cm, or just shy of six feet: half the presidents are shorter than that and half are taller.

10.3 Drawing the histogram

A picture often says more than numbers. The book uses Matplotlib (covered in a later phase) to draw a **histogram**. You do not need to run this code now; just know what it produces.

PYTHON • type this in Colab

```
import matplotlib.pyplot as plt
plt.hist(heights)
plt.title('Height Distribution of US Presidents')
plt.xlabel('height (cm)')
plt.ylabel('number')
```



The histogram, with the mean (red dashed) and median (green line) added.

How to read a histogram

- The **x-axis** is cut into **bins** (ranges of heights); `plt.hist` uses 10 equal bins by default.
- The **height of each bar** (y-axis, "number") is how many presidents fall inside that bin.
- The tallest bar covers 181 to 184 cm: that is the most common range, with 13 presidents.

10.4 The mean-versus-median clue

Look at the two numbers: **mean = 180.05, median = 182**. The mean is a little lower. On the histogram the left side has a long tail (bars stretching down to 163 cm) while the right side stops at 193. A few shorter presidents pull the mean **down**, while the median, which only cares about the middle, stays put.

If you see	It hints at
mean below median	a longer tail on the **left** (some small values)
mean above median	a longer tail on the **right** (some large values)

If you see	It hints at
mean about equal to median	a roughly balanced shape

COMMON TRAP The mean-median gap is a **hint**, not proof. Always look at the histogram as well. Comparing these two numbers is a standard first check on any dataset.

✎ PRACTICE PROBLEMS | Part 10: Worked Example: Heights of US Presidents

Q1 PREDICT THE OUTPUT

Predict the mean, median and standard deviation of these exam scores.

PYTHON • type this in Colab

```
import numpy as np
scores = np.array([60, 70, 80, 90, 100])
print(np.mean(scores))
print(np.median(scores))
print(np.std(scores))
```

Q2 THINK IT THROUGH

A dataset has mean 62 and median 55. Which side of its histogram probably has the longer tail, and why?

Q3 THINK IT THROUGH

Which is more robust to outliers, the mean or the median? Say why in one line.

Q4 BUILD IT

Using the `heights` array, find the tallest height and the index of the first president with that height, and the shortest height and its index.

Q5 READ & DECODE

What does the **height of a bar** in the histogram represent? What does the tallest bar tell you?

Q6 CALCULATE

Find the range (max minus min) of the heights using NumPy, in one line.

Q7 THEORY

[3 marks] Explain why the aggregations in this example "reduce the array to a single value", and say what the mean and the standard deviation tell you.

PART 11

11 Revision Zone

Cheat sheets, interview quick-fire, error radar and a final mixed paper

IN THIS PART YOU WILL LEARN

- ✓ One-page function cheat sheet with priorities
- ✓ Topic checklist for Blocks 3 and 4, and what was left out on purpose
- ✓ 20 interview questions with model answers
- ✓ The error radar: mistakes to check before you trust your output
- ✓ A final mixed paper covering everything

11.1 Function cheat sheet

Group	Function / operator	What it does	Example	Priority
Arithmetic	`+ - * /`	element-wise maths (float for `/`)	`x * 2 + 1`	MUST KNOW
	`// %`	floor division / remainder	`i // 4, i % 4`	MUST KNOW
	`**`	power	`x ** 2`	MUST KNOW
	`np.add, np.multiply ...`	the ufuncs behind the operators	`np.add(x, 2)`	LOOK UP
Absolute value	`np.abs`	remove signs; magnitude of complex	`np.abs(y - p)`	MUST KNOW
Trig	`np.sin / cos / tan`	angles in radians	`np.sin(np.pi / 2)`	GOOD TO KNOW
	`np.arcsin / arccos / arctan`	inverse trig (angle back)	`np.arccos(0)`	LOOK UP
	`np.deg2rad / rad2deg`	convert degrees and radians	`np.deg2rad(90)`	GOOD TO KNOW
	`np.isclose`	compare floats safely	`np.isclose(a, b)`	MUST KNOW
Exp / log	`np.exp`	e to the power x	`np.exp(x)`	MUST KNOW
	`np.log / log2 / log10`	natural / base 2 / base 10 log	`np.log(p)`	MUST KNOW
	`np.expm1 / log1p`	exp(x)-1, log(1+x) for tiny x	`np.log1p(x)`	LOOK UP
Aggregate	`np.sum / x.sum()`	total	`x.sum(axis=0)`	MUST KNOW
	`np.mean / median`	average / middle value	`X.mean(axis=0)`	MUST KNOW
	`np.std / np.var`	spread (ddof=1 for sample)	`X.std(axis=0)`	MUST KNOW
	`np.min / np.max`	smallest / largest	`x.max()`	MUST KNOW
	`np.argmin / np.argmax`	index of smallest / largest	`p.argmax(axis=1)`	MUST KNOW
	`np.nanmean, np.nansum ...`	ignore missing values (NaN)	`np.nanmean(d)`	GOOD TO KNOW
	`axis= / keepdims=`	direction to collapse / keep size 1	`M.sum(axis=1, keepdims=True)`	MUST KNOW

11.2 Topic checklist

Topic from the blocks	Covered in	Done
Block 3: the slowness of loops	Part 1	✓
Block 3: introducing ufuncs and vectorization	Part 2	✓
Block 3: array arithmetic	Part 3	✓
Block 3: absolute value	Part 4	✓
Block 3: trigonometric functions	Part 5	✓
Block 3: exponents and logarithms	Part 6	✓
Block 4: summing, minimum and maximum	Part 7	✓
Block 4: multidimensional aggregates and axis	Part 8	✓
Block 4: other aggregation functions (core set) and NaN-safe versions	Part 9	✓
Block 4: example: heights of US presidents	Part 10	✓

Left out on purpose

These parts of the two blocks were skipped to keep the notes focused. Nothing here blocks you from the next chapters.

Skipped	Why	When it returns
`scipy.special` (gamma, beta, erf)	rarely typed by hand at this stage; search for it when needed	Statistics phase (distributions)
Ufunc `out=` argument	memory saving trick for huge arrays	in-place operations in PyTorch
`ufunc.reduce`, `accumulate`	you will use `np.sum` and `np.cumsum` instead	idea reappears in `axis`
`np.multiply.outer`	outer product of two arrays	broadcasting (next chapter), linear algebra
`np.prod`, `np.percentile`, `np.any`, `np.all`	less common aggregates	boolean masks, data analysis

11.3 Interview quick-fire: 20 questions with model answers

#	Question	Model answer
1	Why are Python loops over arrays slow?	Dynamic typing: type check, function lookup and object creation on every iteration. The arithmetic is not the bottleneck.
2	What is a ufunc?	A NumPy function that applies an operation to every element of an array using fast compiled code.
3	What is vectorization?	Replacing a Python loop with an operation on the whole array so the loop runs in compiled code.
4	Is `x + 2` different from `np.add(x, 2)`?	No. The operator is a wrapper that calls the `np.add` ufunc.
5	What does `/` return for two ints in NumPy?	Always a float; use `//` for floor division that keeps ints.
6	What is `-7 // 2`? What is `-7 % 3`?	-4 (floor rounds down) and 2 (remainder takes the divisor's sign).
7	Which unit is bigger, ms or µs?	ms. 1 s = 1000 ms and 1 ms = 1000 µs.
8	Which angle unit do NumPy trig functions use?	Radians. Use `np.deg2rad` to convert from degrees.
9	Why is `np.sin(np.pi)` not exactly 0?	Floating-point precision: np.pi is an approximation. It is 1.22e-16.
10	How do you safely compare floats?	`np.isclose(a, b)` instead of `==`.

#	Question	Model answer
11	What is `np.log`: base 10 or e?	Base e (natural log). For base 10 use `np.log10`.
12	Why is `-np.log(p)` used as a loss?	It is tiny when the correct class gets a high probability and huge when it gets a low one (cross-entropy).
13	Which two ufuncs build softmax?	`np.exp` (make scores positive) and a division by their sum (make them add to 1).
14	What is an aggregation?	An operation that collapses many values into one, such as sum, mean or max.
15	What does the `axis` argument mean?	The dimension that is collapsed (disappears) in the result.
16	Shape of `X.mean(axis=0)` for X of shape (1000, 20)?	(20,): one mean per feature.
17	Difference between `max` and `argmax`?	`max` gives the largest value; `argmax` gives the index where it is.
18	Why prefer `np.sum` over built-in `sum` for arrays?	Faster (compiled, one type) and `np.sum` understands multiple dimensions and axis.
19	Mean versus median: which resists outliers?	The median.
20	What do the NaN-safe versions do?	Functions such as `np.nanmean` ignore missing values (NaN), which would otherwise turn the whole result into NaN.

11.4 The error radar: check these before you trust your output

#	Mistake	How to spot / fix it
1	Writing a Python loop over a big array	Replace it with a vectorized expression
2	Comparing times without converting units	Convert to one unit: 1 s = 1000 ms = 1,000,000 µs
3	Expecting `/` to return ints	`/` is always float; use `//` for whole numbers
4	Forgetting floor division rounds down	`-7 // 2` is -4, not -3
5	Reading `-(x) ** 2` as a positive square	`**` binds tighter than the leading minus
6	Passing degrees to `np.sin`	Convert with `np.deg2rad` first
7	Using `==` on floats	Use `np.isclose`
8	Assuming `np.log` is base 10	It is natural log; base 10 is `np.log10`
9	Taking `np.log` of 0 or negatives	You get -inf or nan; clip probabilities first
10	Naive softmax on big numbers	`np.exp` overflows: subtract the max first
11	abs of the most negative integer	Wraps around silently; use a larger dtype
12	Using built-in `sum`, `min`, `max` on arrays	Use `np.sum`, `x.sum()` and friends
13	Mixing up the axes	`axis` names the dimension that disappears; check `.shape`
14	Writing a shape as `[1, 3]` instead of `(3,)`	Shapes are tuples; a collapsed axis is removed, not replaced by 1
15	Using `max` when you want `argmax`	Value versus index
16	One NaN in the data	`np.mean` returns nan; use `np.nanmean`
17	Different std from different libraries	Check `ddof` (0 for population, 1 for sample)

✎ PRACTICE PROBLEMS | Final Mixed Paper (Blocks 3 and 4)

Q1 PREDICT THE OUTPUT

Predict every printed value.

PYTHON • type this in Colab

```python
import numpy as np
x = np.array([[1, 2, 3],
              [4, 5, 6]])
print(x.sum(axis=0))
print(x.mean(axis=1))
print(x.max())
print(x ** 2 % 5)
```

Q2 FIND THE BUGS

This code is both slow and wrong: the answer should be `[0.5 1.0 1.5]`. Find every problem and fix it.

PYTHON • type this in Colab

```python
import numpy as np
a = np.array([1, 2, 3])
for i in range(len(a)):
    a[i] = a[i] / 2
print(a)
```

Q3 BUILD IT

Given `y_true = [3, 5, 2, 7]` and `y_pred = [2, 5, 4, 10]`, compute the Mean Absolute Error and the Mean Squared Error.

Q4 BUILD IT

Compute softmax for scores `[1, 2, 3]` and the predicted class using `argmax`.

Q5 CALCULATE

For `X` of shape (500, 10), give the shape of: `X.mean(axis=0)`, `X.sum(axis=1)`, `X.argmax(axis=1)` and `X.max()`.

Q6 PREDICT THE OUTPUT

Predict the three lines.

PYTHON • type this in Colab

```python
import numpy as np
print(np.round(np.sin(np.pi / 6), 2))
print(np.log(np.e))
print(np.isclose(np.exp(np.log(5)), 5))
```

Q7 THINK IT THROUGH

A dataset has a missing value, and `X.mean(axis=0)` returns `nan` for one feature. Why? How do you get the usable means?

PYTHON • type this in Colab

```python
import numpy as np
X = np.array([[1, 2.0],
              [3, np.nan],
              [5, 6.0]])
```

Q8 THEORY

[5 marks] Explain with examples: (i) ufunc and vectorization against a loop, (ii) the `axis` argument, (iii) mean versus median.

ANSWER KEY

HOW TO USE

HOW TO USE. Try every question yourself first. Then compare. If your answer is different, do not just copy mine: find *why* the difference exists. That is where the learning happens.

Every output shown below was produced by really running the code (NumPy 2.4.4).

Answers: Part 1: The Slowness of Loops

Q1 THEORY

CPython is dynamically typed, so on **every iteration** it must (1) check the type of the element, (2) look up the correct routine for that type, (3) do the maths, and (4) wrap the result in a new Python object.

Steps 1, 2 and 4 are overhead repeated a million times. The arithmetic itself (step 3) is tiny.

Q2 THINK IT THROUGH

The friend is wrong. A faster CPU makes the multiplication a little quicker, but the multiplication was never the problem.

The slow part is the type checking, function lookup and object creation that Python repeats for each of the 10 million elements. The real fix is to move the loop into compiled code, where the type is known once.

Q3 READ & DECODE

Only the division `1.0 / x` is arithmetic. The overhead is: checking the type of `values[i]`, looking up the division routine for that type, and creating a new Python object for the result before storing it in `output[i]`.

Q4 CALCULATE

SOLUTION CODE

```
loop_ms = 2.61 * 1000
vec_ms = 2.54
print(loop_ms / vec_ms)
```

▶ OUTPUT

```
1027.55905511811
```

2.61 s = 2610 ms, and 2610 / 2.54 is about 1028. So roughly **1000 times faster**. Without converting the units you would wrongly think the two are almost equal.

Q5 PREDICT THE OUTPUT

▶ OUTPUT

```
[0.5  0.25 0.2  0.1 ]
```

`np.empty(len(values))` creates a **float64** array by default, so every stored result is a float. (Also, 1.0 / 2 is a float division.)

Q6 BUILD IT

SOLUTION CODE

```
import time
import numpy as np

big = np.random.default_rng(0).integers(1, 100, size=1_000_000)

t0 = time.perf_counter()
out = np.empty(len(big))
for i in range(len(big)):
    out[i] = 1.0 / big[i]
t_loop = time.perf_counter() - t0

t0 = time.perf_counter()
vec = 1.0 / big
t_vec = time.perf_counter() - t0

print("faster by at least 100x:", t_loop / t_vec > 100)
```

▶ OUTPUT

```
faster by at least 100x: True
```

The exact ratio changes from machine to machine (and from run to run), so the program prints a True/False check. On almost every computer the vectorized line wins by two to three orders of magnitude.

Q7 THEORY

NumPy arrays hold one type only, so NumPy checks the type **once** and then runs a pre-compiled C loop over the whole block of memory. The loop still exists, but it now runs in compiled code instead of in the Python interpreter.

Because the C loop already knows the type, it skips the per-element type check, function lookup and object creation. Those repeated costs were what made the Python loop slow.

Answers: Part 2: Introducing Ufuncs

Q1 THEORY

Ufunc: a NumPy function that applies an operation to every element of an array quickly, using compiled code.

Vectorization: writing the operation on the whole array (for example `1.0 / values`) instead of looping in Python.

Why faster: an array has one fixed type, so the type is checked once and a compiled C loop runs over the whole block, instead of Python repeating a type check on each element.

Q2 PREDICT THE OUTPUT

▶ OUTPUT

```
[1.   0.5  0.25 0.2 ]
```

A scalar with an array: each element is divided into 1.0, giving 1/1, 1/2, 1/4, 1/5. The result is float.

Q3 BUILD IT

SOLUTION CODE

```
import numpy as np
a = np.array([1, 2, 3, 4, 5])
print((a ** 2) + 10)
```

▶ OUTPUT

```
[11 14 19 26 35]
```

The squares are 1, 4, 9, 16, 25 and adding 10 gives the output above. The loop version would print **floats** (`[11. 14. 19. 26. 35.]`) because `np.empty` makes a float64 array; the vectorized one keeps the integer type.

Q4 PREDICT THE OUTPUT

▶ **OUTPUT**

```
[ 0 11 24 39]
```

Position by position: 0×10, 1×11, 2×12, 3×13.

Q5 PREDICT THE OUTPUT

▶ **OUTPUT**

```
[[  1   3   9]
 [ 27  81 243]]
```

`x` is `[[0 1 2], [3 4 5]]`, and `3 ** x` raises 3 to each element: 3^0, 3^1, 3^2 / 3^3, 3^4, 3^5.

Q6 FIND THE BUGS

▶ **OUTPUT**

```
ValueError: operands could not be broadcast together with shapes (3,) (4,)
```

The arrays have shapes (3,) and (4,). A ufunc between two arrays pairs elements by position, and the fourth element of the second array has no partner. The shapes must match (or be compatible for broadcasting, covered next chapter).

Q7 THINK IT THROUGH

SOLUTION CODE

```python
import numpy as np
a = np.array([2, 4, 6, 8])
b = a * 0.5 + 1
print(b)
```

▶ **OUTPUT**

```
[2. 3. 4. 5.]
```

Replace the loop with the vectorized expression `b = a * 0.5 + 1`. It is shorter, clearer and far faster on large arrays, because the loop now runs in compiled code.

Q8 READ & DECODE

It raises 2 to the power of **every** element of `x` and returns a new 3×3 array. The operator is `**`, which is a wrapper for the ufunc `np.power`. The single number 2 is applied against every element (scalar with array).

Answers: Part 3: Array Arithmetic

Q1 PREDICT THE OUTPUT

▶ **OUTPUT**

```
[2 3 5 6]
[2 3 0 1]
[-1. -2. -3. -4.]
```

`x // 4`: 2, 3, 5, 6 (whole-number part). `x % 4`: remainders 2, 3, 0, 1. `x / 5` gives 2, 3, 4, 5 as floats, the minus flips them, then +1: -1, -2, -3, -4.

Q2 READ & DECODE

(a) `np.multiply` (b) `np.negative` (c) `np.floor_divide` (d) `np.mod` (e) `np.power`.

Note that `-` has two meanings: with two operands it is `np.subtract`, with one operand it is `np.negative`.

Q3 PREDICT THE OUTPUT

▶ **OUTPUT**

```
[ 3  9 17]
```

Powers first, then multiplication, then addition and subtraction: element 1: 1 + 3 - 1 = 3; element 2: 4 + 6 - 1 = 9; element 3: 9 + 9 - 1 = 17.

Q4 CALCULATE

SOLUTION CODE

```
print(7 // 2, -7 // 2, 7 % 3, -7 % 3)
```

▶ OUTPUT

```
3 -4 1 2
```

(a) 3. (b) **-4** because floor rounds down, not towards zero. (c) 1. (d) **2**, because the remainder takes the sign of the divisor: -7 = (-3) × 3 + 2.

Q5 PREDICT THE OUTPUT

▶ OUTPUT

```
float64
int64
float64
```

`/` always returns float; `//` on ints stays int; adding a float to ints upcasts the whole result to float.

Q6 BUILD IT

SOLUTION CODE

```
import numpy as np
samples = np.arange(10)
print(samples // 4)
print(samples % 4)
```

▶ OUTPUT

```
[0 0 0 0 1 1 1 1 2 2]
[0 1 2 3 0 1 2 3 0 1]
```

`//` gives the batch number and `%` gives the position inside it. Samples 8 and 9 form a short last batch (batch 2, positions 0 and 1).

Q7 FIND THE BUGS

▶ OUTPUT

```
[0.5 1.  1.5]
```

SOLUTION CODE

```
import numpy as np
print(np.array([1, 2, 3]) // 2)
```

▶ OUTPUT

```
[0 1 1]
```

`/` always returns floats. Use floor division `//` to keep whole numbers.

Q8 THINK IT THROUGH

None in result: `+` is only a convenient wrapper that calls the `np.add` ufunc. In real code write `x + 2` because it is shorter and easier to read; use the named ufunc when you need extra features such as the `out` argument.

Q9 BUILD IT

SOLUTION CODE

```
import numpy as np
c = np.array([0, 37, 100])
print(c * 9 / 5 + 32)
```

▶ OUTPUT

```
[ 32.   98.6 212. ]
```

One expression converts every temperature at once. The `/` makes the result float, which is why you see decimal points.

Answers: Part 4: Absolute Value

Q1 PREDICT THE OUTPUT

▶ OUTPUT

```
[5.  3.  0.  1.5]
```

Each negative number loses its sign. The array contains a float (-1.5), so the whole result is float.

Q2 CALCULATE

SOLUTION CODE

```
import numpy as np
print(np.abs(np.array([5 + 12j, 6 - 8j])))
```

▶ OUTPUT

```
[13. 10.]
```

`5 + 12j`: √(25 + 144) = √169 = 13. `6 - 8j`: √(36 + 64) = √100 = 10. Both are the length of the hypotenuse of a right triangle (a vector's length).

Q3 BUILD IT

SOLUTION CODE

```
import numpy as np
y_true = np.array([3, 5, 2, 7])
y_pred = np.array([2, 5, 4, 10])
errors = y_true - y_pred
print(errors)
print(np.abs(errors))
print(np.mean(np.abs(errors)))
```

▶ OUTPUT

```
[ 1  0 -2 -3]
[1 0 2 3]
1.5
```

Errors are 1, 0, -2, -3. Absolute values: 1, 0, 2, 3. Mean: 6 / 4 = **1.5**. The model is off by 1.5 units on average.

Q4 THINK IT THROUGH

The average of +5 and -5 is 0, which would mean "no error" although both predictions are 5 units wrong. Positive and negative errors cancel out.

`np.abs` removes the sign first, so both count as 5 and the mean correctly becomes 5.

Q5 FIND THE BUGS

SOLUTION CODE

```
import numpy as np
errors = np.array([4, -3, 0, -7])
print(np.abs(errors))
```

▶ OUTPUT

```
[4 3 0 7]
```

`np.abs[errors]` uses square brackets, which are for indexing; a ufunc must be **called** with round brackets: `np.abs(errors)`.

`errors.abs()` fails because arrays have no `.abs()` method. `abs` is a function (`abs(errors)` or `np.abs(errors)`), unlike `.sum()` or `.max()` which are methods.

Q6 PREDICT THE OUTPUT

▶ OUTPUT

```
[-128]
```

The true answer +128 does not fit in an int8 (maximum 127), so it wraps around to -128 silently. Convert to a larger dtype first (`astype(np.int16)`) if values can reach the type's minimum.

Q7 THEORY

MAE = the average size of the errors, ignoring their direction: mean of |y_true − y_pred|. NumPy: `np.mean(np.abs(y_true - y_pred))`.

Another use: L1 regularisation, `np.sum(np.abs(weights))`, which penalises large weights.

Answers: Part 5: Trigonometric Functions

Q1 PREDICT THE OUTPUT

▶ OUTPUT

```
1.0
1.0
True
```

sin(π/2) = 1 (the peak of the sine wave). cos(0) = 1. `np.isclose` is `True` because `sin(np.pi)` is within a tiny tolerance of 0.

Q2 THINK IT THROUGH

A computer stores numbers with limited precision, so `np.pi` is a very close approximation of π, not π itself. The sine of a slightly-off value is slightly off 0. The result is 0.00000000000000122, effectively zero but not exactly.

Q3 PREDICT THE OUTPUT

▶ OUTPUT

```
[ 0.   2.5  5.   7.5 10. ]
```

5 points include both ends, so there are 4 gaps of (10 − 0) / 4 = 2.5.

Q4 CALCULATE

SOLUTION CODE

```
import numpy as np
print(np.deg2rad([0, 90, 180, 360]))
print(np.rad2deg(3.14159))
```

▶ OUTPUT

```
[0.         1.57079633 3.14159265 6.28318531]
179.9998479605043
```

Degrees multiply by π/180. 90 gives 1.5708 (π/2), 180 gives 3.1416 (π), 360 gives 6.2832 (2π). 3.14159 radians is very nearly 180 degrees.

Q5 FIND THE BUGS

▶ OUTPUT

```
0.8939966636005579
```

SOLUTION CODE

```
import numpy as np
print(np.sin(np.deg2rad(90)))
```

▶ OUTPUT

```
1.0
```

NumPy reads 90 as **radians**, not degrees. Convert with `np.deg2rad(90)` first.

Q6 PREDICT THE OUTPUT

▶ OUTPUT

```
[0.         1.57079633 3.14159265]
```

`arccos` returns the angle whose cosine is the given value: cos(0) = 1, cos(π/2) = 0, cos(π) = -1.

Q7 THEORY

Floats are stored approximately, so values that should be equal can differ in the last few digits. Example: `np.sin(np.pi) == 0` is `False` because the result is 1.22e-16.

Use `np.isclose(a, b)` (or `np.allclose` for arrays), which tests equality within a small tolerance.

Answers: Part 6: Exponents and Logarithms

Q1 CALCULATE

SOLUTION CODE

```
import numpy as np
print(np.log2(8), np.log10(1000), np.log(1), np.exp(0))
```

▶ OUTPUT

```
3.0 3.0 0.0 1.0
```

$2^3 = 8$, so 3. $10^3 = 1000$, so 3. $e^0 = 1$, so the natural log of 1 is 0. And $e^0 = 1$.

Q2 THINK IT THROUGH

SOLUTION CODE

```
import numpy as np
print(np.log(100))
print(np.log10(100))
```

▶ OUTPUT

```
4.605170185988092
2.0
```

It returns about 4.605, which is larger than 2. `np.log` is the **natural** log (base e); the student was thinking of base 10, which is `np.log10(100) = 2`.

Q3 BUILD IT

SOLUTION CODE

```
import numpy as np
x = np.array([-2, 0, 2])
print(1 / (1 + np.exp(-x)))
```

▶ OUTPUT

```
[0.11920292 0.5        0.88079708]
```

The formula $1 / (1 + e^{-x})$ is vectorized: it works on the whole array at once. sigmoid(0) is exactly 0.5.

Q4 PREDICT THE OUTPUT

▶ OUTPUT

```
[   2.    4.    8. 1024.]
[1. 3. 9.]
```

`np.exp2` is 2^x: 2, 4, 8, 1024. `np.power(3.0, ...)` is 3^0, 3^1, 3^2 = 1, 3, 9 (floats because of the 3.0).

Q5 PREDICT THE OUTPUT

SOLUTION CODE

```
import numpy as np
e = np.exp(np.array([2.0, 2.0, 2.0]))
print(e / e.sum())
```

▶ OUTPUT

```
[0.33333333 0.33333333 0.33333333]
```

Equal scores must give equal probabilities, so each is 1/3 = 0.3333. The probabilities always add up to 1.

Q6 THINK IT THROUGH

SOLUTION CODE

```
import numpy as np
print(-np.log(0.5))
print(-np.log(0.01))
```

▶ OUTPUT

```
0.6931471805599453
4.605170185988091
```

-log(0.5) = 0.693. log of a number close to 0 is a big negative number, so its negative is large: -log(0.01) = 4.605. The loss heavily punishes a model that is confident and wrong.

Q7 FIND THE BUGS

▶ OUTPUT

```
[0.35667494        inf 1.2039728 ]
```

SOLUTION CODE

```
import numpy as np
probs = np.array([0.7, 0.0, 0.3])
safe = np.clip(probs, 1e-12, 1)
print(-np.log(safe))
```

▶ OUTPUT

```
[ 0.35667494 27.63102112  1.2039728 ]
```

`log(0)` is `-inf`, so the loss becomes `inf`. Clip probabilities away from 0 with `np.clip(probs, 1e-12, 1)` before taking the log.

Q8 CALCULATE

SOLUTION CODE

```
import numpy as np
x = 1e-10
print(np.log(1 + x))
print(np.log1p(x))
```

▶ OUTPUT

```
1.000000082690371e-10
9.999999999500001e-11
```

`np.log1p(x)` matches the true value; `np.log(1 + x)` has lost digits because forming `1 + x` first throws tiny digits away.

Q9 THEORY

A log answers "what power gives this number?": if $b^y = x$ then $\log_b(x) = y$. `np.exp` and `np.log` undo each other.

`np.log` is base e (natural log); `np.log10` is base 10.

ML uses: softmax and sigmoid use `np.exp`; cross-entropy loss uses `np.log`.

Answers: Part 7: sum, min and max

Q1 PREDICT THE OUTPUT

▶ OUTPUT

```
16
```

The second argument of the built-in `sum` is the **starting value**, so the total is 10 + 1 + 2 + 3 = 16.

Q2 THINK IT THROUGH

`arr.sum()`. The built-in `sum` loops in Python and checks the **type of every element** as it goes; with 5 million elements that repeated overhead is huge.

`arr.sum()` is compiled code: arrays are homogeneous (one type), so the type is checked once and one compiled loop adds everything.

Q3 BUILD IT

SOLUTION CODE

```
import numpy as np
x = np.array([4, 9, 1, 7])
print(x.min(), x.max(), x.sum())
```

▶ OUTPUT

```
1 9 21
```

`x.min()`, `x.max()`, `x.sum()` are shorthand for `np.min(x)`, `np.max(x)`, `np.sum(x)`.

Q4 PREDICT THE OUTPUT

▶ OUTPUT

```
[3 7]
[5 7]
```

`np.sum(a, 1)`: the 1 is the **axis**, so each row is added: 1+2 = 3, 3+4 = 7. The built-in `sum(a, 1)`: the 1 is the **start value** and it adds whole rows to it: 1 + [1, 2] + [3, 4] = [5, 7].

Q5 CALCULATE

SOLUTION CODE

```
builtin_us = 89.9 * 1000
numpy_us = 521
print(builtin_us / numpy_us)
```

▶ OUTPUT

```
172.552783109405
```

Convert to the same unit first: 89.9 ms = 89,900 µs. Then 89,900 / 521 is about 172, so NumPy is roughly **170 times faster**.

Q6 FIND THE BUGS

▶ OUTPUT

```
15
```

SOLUTION CODE

```
import numpy as np
a = np.array([4, 9, 1])
print(np.sum(a))
```

▶ OUTPUT

```
14
```

With the built-in `sum`, the 1 is the starting value, so the answer is 1 + 14 = 15. Use `np.sum(a)` (or `a.sum()`), which has no such argument.

Q7 THEORY

Faster: (1) NumPy runs the loop in **compiled code**; (2) arrays hold **one type**, so the type is checked once, not for every element.

Difference: the optional second argument means a start value for built-in `sum` but the axis for `np.sum`; `np.sum` is also aware of multiple dimensions.

Answers: Part 8: Multidimensional Aggregates and axis

Q1 PREDICT THE OUTPUT

▶ **OUTPUT**

```
[ 3  7 11] (3,)
[12  9] (2,)
21
```

`axis=0` collapses the 2 rows, leaving one total per column: 3, 7, 11, shape (3,). `axis=1` collapses the 3 columns, leaving one per row: 12, 9, shape (2,). The grand total is 21. Shapes are **tuples** such as `(3,)`, not `[1, 3]`.

Q2 CALCULATE

SOLUTION CODE

```
import numpy as np
print(np.zeros((2, 3)).sum(axis=0).shape)
print(np.zeros((2, 3)).sum(axis=1).shape)
print(np.zeros((1000, 20)).sum(axis=0).shape)
print(np.zeros((32, 64, 64, 3)).sum(axis=0).shape)
```

▶ **OUTPUT**

```
(3,)
(2,)
(20,)
(64, 64, 3)
```

Delete the named axis from the shape: (a) (3,) (b) (2,) (c) (20,) (d) (64, 64, 3).

Q3 BUILD IT

SOLUTION CODE

```
import numpy as np
X = np.random.default_rng(0).random((1000, 20))
m = X.mean(axis=0)
print(m.shape)
```

▶ **OUTPUT**

```
(20,)
```

`X.mean(axis=0)`: collapse the 1000 samples, keep the 20 features. The result has shape **(20,)**.

Q4 PREDICT THE OUTPUT

▶ **OUTPUT**

```
[9 8 7]
[4 1 2]
```

`max(axis=1)` is the largest in each row: 9, 8, 7. `min(axis=0)` is the smallest in each column: 4, 1, 2. Read the function name (max or min) before you compute.

Q5 FIND THE BUGS

▶ **OUTPUT**

```
[ 6 15]
```

SOLUTION CODE

```
import numpy as np
X = np.array([[1, 2, 3],
              [4, 5, 6]])
print(X.sum(axis=0))
```

▶ **OUTPUT**

```
[5 7 9]
```

`axis=1` collapses the columns and gives one total per **row**. For column totals collapse the rows with `axis=0`.

Q6 READ & DECODE

(a) the mean of each feature (one value per column). (b) the largest value in each sample (one value per row). (c) the total of every value as one number.

Q7 THINK IT THROUGH

`axis` names the dimension that is **collapsed**, not the one returned. `axis=0` collapses the rows (going down each column), so what survives is one value per column.

Q8 BUILD IT

SOLUTION CODE

```
import numpy as np
X = np.array([[170, 65], [160, 50], [180, 80], [175, 75]])
Z = (X - X.mean(axis=0)) / X.std(axis=0)
print(np.allclose(Z.mean(axis=0), 0))
```

▶ OUTPUT

```
True
```

Subtract the per-feature mean, divide by the per-feature std. The means are not exactly 0 because of floating-point rounding, so use `np.allclose` instead of `==`.

Q9 THEORY

`axis` is the dimension that is collapsed. Example: for `M` of shape (3, 4), `M.sum(axis=0)` adds down the rows and returns 4 column totals.

Shape rule: the named axis is removed from the shape, so (3, 4) becomes (4,) for `axis=0` and (3,) for `axis=1`.

Answers: Part 9: The Core Aggregation Functions

Q1 CALCULATE

SOLUTION CODE

```
import numpy as np
y = np.array([7, 3, 9, 3, 5])
print(np.mean(y), np.median(y), np.argmax(y), np.argmin(y))
```

▶ OUTPUT

```
5.4 5.0 2 1
```

Mean: 27 / 5 = 5.4. Sorted: 3, 3, 5, 7, 9, so the median is 5. The maximum 9 is at index 2. The minimum 3 appears at index 1 and 3; `argmin` returns the **first**, index 1.

Q2 CALCULATE

SOLUTION CODE

```
import numpy as np
x = np.array([1, 3, 5, 7])
print(np.var(x))
print(np.std(x))
```

▶ OUTPUT

```
5.0
2.23606797749979
```

Mean = 4. Distances: -3, -1, 1, 3. Squares: 9, 1, 1, 9 (sum 20). Variance = 20 / 4 = 5. Standard deviation = √5 = 2.236.

Q3 THINK IT THROUGH

The median. One outlier (1000) pulls the mean far away from where almost all the data sits, while the median only cares about the middle position and barely moves.

Q4 BUILD IT

SOLUTION CODE

```
import numpy as np
probs = np.array([[0.2, 0.5, 0.3],
                  [0.6, 0.3, 0.1],
                  [0.1, 0.1, 0.8]])
print(probs.argmax(axis=1))
```

▶ OUTPUT

```
[1 0 2]
```

`probs.argmax(axis=1)` collapses the classes of each row and returns the **position** of the biggest probability: class 1, class 0, class 2.

Q5 PREDICT THE OUTPUT

▶ OUTPUT

```
nan
3.0
6.0
4.0
```

One NaN ruins `np.mean` (result nan). The `nan` versions skip it: (4 + 2) / 2 = 3, total 6, maximum 4.

Q6 FIND THE BUGS

▶ OUTPUT

```
[0.9 0.7]
```

SOLUTION CODE

```
import numpy as np
probs = np.array([[0.1, 0.9], [0.7, 0.3]])
pred = np.argmax(probs, axis=1)
print(pred)
```

▶ OUTPUT

```
[1 0]
```

`max` returns the biggest **value**; the class label is the **position** of that value, which is what `argmax` returns.

Q7 THINK IT THROUGH

SOLUTION CODE

```
import numpy as np
x = np.array([2, 4, 4, 4, 5, 5, 7, 9])
print(np.std(x))
print(np.std(x, ddof=1))
```

▶ OUTPUT

```
2.0
2.138089935299395
```

`np.std` divides by n by default; `ddof=1` divides by n-1 (the sample formula). Libraries differ in their default, so always check which one you are getting.

Q8 READ & DECODE

(a) the standard deviation of each column, ignoring missing values. (b) for each row, the index of its largest value. (c) the variance (average squared distance from the mean) of all of `x`.

Q9 THEORY

For `[3, 9, 4]`, `max` is 9 (the value) and `argmax` is 1 (its index).

ML uses: predicted class of each sample from a probability table; picking the most likely next token in a language model (greedy decoding).

Answers: Part 10: Worked Example: Heights of US Presidents

Q1 PREDICT THE OUTPUT

▶ OUTPUT

```
80.0
80.0
14.142135623730951
```

Mean: 400 / 5 = 80. Median: the middle value, 80. Distances: -20, -10, 0, 10, 20; squares sum to 1000; variance 200; std = √200 = 14.14.

Q2 THINK IT THROUGH

The **right** side. Large values pull the mean up above the median, so there must be a few unusually big values.

Q3 THINK IT THROUGH

The median: it depends only on the middle position of the sorted data, so one extreme value cannot move it much, while it directly changes the sum behind the mean.

Q4 BUILD IT

SOLUTION CODE

```
import numpy as np
heights = np.array([189, 170, 189, 163, 183, 171, 185, 168, 173, 183, 173, 173,
                    175, 178, 183, 193, 178, 173, 174, 183, 183, 168, 170, 178,
                    182, 180, 183, 178, 182, 188, 175, 179, 183, 193, 182, 183,
                    177, 185, 188, 188, 182, 185, 191, 182])
print(heights.max(), heights.argmax())
print(heights.min(), heights.argmin())
```

▶ OUTPUT

```
193 15
163 3
```

`max` and `min` give the values; `argmax` and `argmin` give the positions (and report the first one when a value repeats, as 193 does).

Q5 READ & DECODE

The height of a bar is the **number of presidents** whose height falls inside that bin. The tallest bar marks the most common range of heights (181 to 184 cm).

Q6 CALCULATE

SOLUTION CODE

```
import numpy as np
heights = np.array([189, 170, 189, 163, 183, 171, 185, 168, 173, 183, 173, 173,
                    175, 178, 183, 193, 178, 173, 174, 183, 183, 168, 170, 178,
                    182, 180, 183, 178, 182, 188, 175, 179, 183, 193, 182, 183,
                    177, 185, 188, 188, 182, 185, 191, 182])
print(heights.max() - heights.min())
```

▶ OUTPUT

```
30
```

193 - 163 = 30 cm.

Q7 THEORY

Each aggregation takes all 44 numbers and returns one number summarising them.

The **mean** is the typical value (180 cm). The **standard deviation** is how spread out the data is around it (about 7 cm).

Answers: Final Mixed Paper (Blocks 3 and 4)

Q1 PREDICT THE OUTPUT

▶ OUTPUT

```
[5 7 9]
[2. 5.]
6
[[1 4 4]
 [1 0 1]]
```

`sum(axis=0)` adds down the columns: 5, 7, 9. `mean(axis=1)` averages each row: 2 and 5 (floats). `max()` is 6. `x ** 2 % 5` squares first (1, 4, 9 / 16, 25, 36) then takes the remainder by 5: 1, 4, 4 / 1, 0, 1.

Q2 FIND THE BUGS

▶ OUTPUT

```
[0 1 1]
```

SOLUTION CODE

```
import numpy as np
a = np.array([1, 2, 3])
b = a / 2
print(b)
```

▶ OUTPUT

```
[0.5 1.  1.5]
```

Bug 1 (speed): a Python loop over an array. Bug 2 (correctness): `a` is an **integer** array, so storing 0.5 truncates it to 0 silently. The vectorized `a / 2` is faster and returns a new float array.

Q3 BUILD IT

SOLUTION CODE

```
import numpy as np
y_true = np.array([3, 5, 2, 7])
y_pred = np.array([2, 5, 4, 10])
err = y_true - y_pred
print(np.mean(np.abs(err)))
print(np.mean(err ** 2))
```

▶ OUTPUT

```
1.5
3.5
```

Errors: 1, 0, -2, -3. MAE = (1+0+2+3)/4 = 1.5. MSE = (1+0+4+9)/4 = 3.5. Squaring punishes large errors more than abs does.

Q4 BUILD IT

SOLUTION CODE

```
import numpy as np
s = np.array([1.0, 2.0, 3.0])
e = np.exp(s - s.max())
p = e / e.sum()
print(p)
print(p.sum())
print(p.argmax())
```

▶ OUTPUT

```
[0.09003057 0.24472847 0.66524096]
0.9999999999999999
2
```

Exponentiate, divide by the total: the probabilities add up to 1 and the biggest score (3, at index 2) gets the biggest probability. `argmax` returns the index 2. Subtracting the max first keeps the answer the same but avoids overflow.

Q5 CALCULATE

SOLUTION CODE

```
import numpy as np
X = np.zeros((500, 10))
print(X.mean(axis=0).shape)
print(X.sum(axis=1).shape)
print(X.argmax(axis=1).shape)
print(np.shape(X.max()))
```

▶ OUTPUT

```
(10,)
(500,)
(500,)
()
```

`axis=0` removes the 500: (10,). `axis=1` removes the 10: (500,) for both `sum` and `argmax`. `X.max()` is a single number, shape () .

Q6 PREDICT THE OUTPUT

▶ OUTPUT

```
0.5
1.0
True
```

sin(30 degrees) = 0.5. The natural log of e is 1. `exp(log(5))` is 4.999999999999999, so `np.isclose` is the safe test and it returns True.

Q7 THINK IT THROUGH

SOLUTION CODE

```
import numpy as np
X = np.array([[1, 2.0], [3, np.nan], [5, 6.0]])
print(X.mean(axis=0))
print(np.nanmean(X, axis=0))
```

▶ OUTPUT

```
[ 3. nan]
[3. 4.]
```

One NaN poisons every ordinary aggregate that touches it. `np.nanmean(X, axis=0)` ignores NaN and averages the remaining values (here (2 + 6) / 2 = 4 for the second feature).

Q8 THEORY

(i) `1.0 / values` is one ufunc call: NumPy checks the type once and runs a compiled loop, instead of Python repeating the type check for every element (about 1000 times faster in the book's test).

(ii) `axis` is the dimension that is collapsed. For M of shape (3, 4): `M.sum(axis=0)` gives 4 column totals, `M.sum(axis=1)` gives 3 row totals.

(iii) The mean is the average and is dragged by outliers; the median is the middle value and resists them. For [2, 4, 4, 4, 5, 5, 7, 1000] the mean is about 129 while the median stays 4.5.